ECC 스킬 완전 가이드 (142개)

작성일: 2026-03-26 **대상**: CORTHEX v2 프로젝트 — 비개발자를 위한 설명서

이 문서 사용법

ECC(Everything Claude Code) 스킬은 **Claude Code에 설치하는 앱**이라고 생각하시면 됩니다.

스마트폰에 카메라 앱, 계산기 앱, 지도 앱이 있듯이, Claude Code에도 "코드 리뷰 스킬", "보안 검사 스킬", "디자인 스킬" 같은 것들이 있습니다. 각 스킬은 Claude가 특정 작업을 더 잘하도록 전문 지식을 제공합니다.

스킬의 종류는 크게 세 가지입니다:

1. **자동 스킬** — 별도로 실행하지 않아도 알아서 작동합니다 (백그라운드 앱처럼)
2. **명령어 스킬** — /□□□를 입력해서 직접 실행합니다 (앱을 터치해서 여는 것처럼)
3. **참고 스킬** — Claude가 관련 작업을 할 때 자동으로 참조합니다 (사전처럼)

총 142개 스킬이 11개 분류로 나뉘어 있습니다. 아래에서 하나하나 쉽게 설명해 드리겠습니다.

---

1. 자동으로 돌아가는 것 (자율 학습/관리 시스템)

이 스킬들은 사람이 신경 쓰지 않아도 Claude가 스스로 학습하고 관리하는 시스템입니다. 마치 스마트폰이 밤에 자동으로 업데이트되는 것과 같습니다.

---

dream

한 줄 요약: Claude가 "잠자면서" 기억을 정리하는 스킬입니다.

비유: 사람이 자는 동안 뇌가 낮에 배운 것을 정리하듯이, Claude도 세션이 끝나면 배운 것을 정리해서 기억 파일에 저장합니다.

언제 쓰나요? - 24시간마다 자동으로 실행됩니다 - Claude가 작업하다가 배운 교정사항, 결정, 선호도 등을 정리할 때

어떻게 작동하나요? 세션이 끝나면 Claude가 대화 내용을 훑어보면서 "아, 이 사용자는 이런 걸 좋아하는구나", "이 프로젝트에서는 이렇게 하면 안 되는구나" 같은 패턴을 찾아냅니다. 찾아낸 패턴은 MEMORY.md 같은 기억 파일에 자동 저장됩니다. 다음에 대화할 때 이 기억을 읽어서 더 잘 도와줄 수 있습니다.

사용법: 자동으로 실행됨 (Stop hook)

---

continuous-learning

한 줄 요약: 작업 세션에서 반복되는 패턴을 자동으로 배워서 저장하는 스킬입니다.

비유: 요리사가 매일 요리하면서 자연스럽게 자신만의 노하우를 축적하는 것과 같습니다. Claude도 작업하면서 "이 에러는 이렇게 고치면 되더라"를 자동으로 기록합니다.

언제 쓰나요? - 세션이 끝날 때 자동으로 실행됩니다 - 충분히 긴 세션(메시지 10개 이상)일 때만 작동합니다

어떻게 작동하나요? 세션이 끝나면 Claude가 대화를 분석해서 에러 해결법, 사용자 교정사항, 우회 방법 등의 패턴을 감지합니다. 유용한 패턴은 learned 폴더에 스킬 형태로 저장되어, 나중에 비슷한 상황에서 자동으로 참고됩니다.

사용법: 자동으로 실행됨 (Stop hook)

---

continuous-learning-v2

한 줄 요약: continuous-learning의 개선판으로, "본능(instinct)" 단위로 더 세밀하게 학습하는 스킬입니다.

비유: v1이 "오늘 배운 것 일기쓰기"라면, v2는 "핵심 한 줄씩 포스트잇에 적어서 분류하는 것"입니다. 더 작은 단위로, 더 정확하게 배웁니다.

언제 쓰나요? - 매 도구 사용 전후에 자동으로 분석합니다 - 프로젝트별로 별도 저장되어, React 프로젝트에서 배운 것이 Python 프로젝트에 섞이지 않습니다

어떻게 작동하나요? Claude가 도구를 사용할 때마다 작은 분석 에이전트가 뒤에서 "이번 작업에서 배울 게 있나?" 확인합니다. 배울 것이 있으면 신뢰도 점수(0.3~0.9)와 함께 저장합니다. 여러 프로젝트에서 같은 패턴이 발견되면 전역 지식으로 승격됩니다.

사용법: 자동으로 실행됨 (PreToolUse/PostToolUse hook)

---

canary-watch

한 줄 요약: 배포 후 사이트가 잘 되는지 자동으로 감시하는 스킬입니다.

비유: 가게 주인이 CCTV로 가게를 감시하는 것처럼, 웹사이트에 문제가 없는지 자동으로 확인합니다.

언제 쓰나요? - 새 기능을 배포한 직후 - 중요한 코드 변경 후 문제가 없는지 확인하고 싶을 때 - 밤새 사이트를 감시하고 싶을 때

어떻게 작동하나요? 지정한 웹사이트 주소로 접속해서 페이지가 정상인지(200 응답), 에러가 없는지, 성능은 괜찮은지 체크합니다. 정해진 간격으로 반복 체크할 수도 있고, 스테이징 서버와 실서버를 비교할 수도 있습니다.

사용법: `/canary-watch https://□□□□□`

---

continuous-agent-loop

한 줄 요약: Claude가 사람 없이 혼자서 반복 작업을 계속하게 만드는 스킬입니다.

비유: 공장의 자동 조립 라인처럼, Claude가 정해진 작업을 사람 개입 없이 반복합니다.

언제 쓰나요? - 사람이 없어도 Claude가 계속 작업하게 하고 싶을 때 - 코드 작성 > 테스트 > 수정을 자동으로 반복하고 싶을 때 - 여러 에이전트가 동시에 작업하고 합치는 방식이 필요할 때

어떻게 작동하나요? Claude가 "작업 > 검증 > 수정" 루프를 자동으로 반복합니다. 단순 순차 실행부터, 여러 에이전트가 동시에 병렬 작업하는 복잡한 방식까지 다양한 패턴을 지원합니다. 품질 검증 단계를 통과해야 다음으로 넘어가는 안전장치가 있습니다.

사용법: 자동으로 실행됨 (autonomous-loops 패턴의 후속)

---

autonomous-loops

한 줄 요약: Claude가 자율적으로 반복 실행하는 다양한 패턴 모음입니다.

비유: 자동 세탁기의 다양한 코스(빨래, 헹굼, 탈수)처럼, 작업을 자동으로 반복하는 여러 방식이 담겨 있습니다.

언제 쓰나요? - 자동 개발 워크플로우를 만들 때 - 어떤 루프 방식이 내 작업에 맞는지 골라야 할 때

어떻게 작동하나요? 단순한 순차 파이프라인부터, RFC(설계서) 기반 멀티 에이전트 시스템까지 다양한 자동화 패턴을 제공합니다. 프로젝트 성격에 따라 적합한 패턴을 선택해서 사용합니다. continuous-agent-loop이 이 스킬의 후속 버전입니다.

사용법: 프로젝트 설정 시 참고

---

strategic-compact

한 줄 요약: 대화가 너무 길어지기 전에 적절한 타이밍에 기억을 정리하도록 알려주는 스킬입니다.

비유: 책상이 너무 어질러지기 전에 "지금 정리하면 좋겠다"고 알려주는 비서와 같습니다.

언제 쓰나요? - 긴 세션에서 Claude의 기억 공간이 부족해질 때 - 한 작업이 끝나고 다른 작업으로 넘어갈 때 - 자동으로 도구 사용 횟수(기본 50회)를 추적해서 알림을 줍니다

어떻게 작동하나요? Claude가 도구를 몇 번 사용했는지 추적합니다. 일정 횟수를 넘기면 "지금 기억을 정리하면 좋겠습니다"라고 제안합니다. 작업 단계가 바뀌는 논리적 경계에서 정리하면, 중요한 맥락을 잃지 않고 새 작업을 시작할 수 있습니다.

사용법: 자동으로 실행됨 (PreToolUse hook)

---

dmux-workflows

한 줄 요약: 여러 AI 에이전트를 동시에 실행해서 병렬로 작업시키는 스킬입니다.

비유: 요리사 여러 명이 동시에 다른 요리를 만들고, 마지막에 한 식탁에 차리는 것처럼, 여러 Claude를 동시에 작업시키고 결과를 합칩니다.

언제 쓰나요? - 여러 작업을 동시에 진행하고 싶을 때 - "이것도 하고 저것도 해야 하는데" 할 때 분담시키기 - 복잡한 작업을 나눠서 병렬 처리하고 싶을 때

어떻게 작동하나요? dmux라는 도구를 사용해서 터미널 창을 여러 개 열고, 각 창에서 별도의 Claude가 작업합니다. 한 창에서는 인증 기능을, 다른 창에서는 테스트를, 또 다른 창에서는 문서를 동시에 작성합니다. 결과를 합칠 때는 m 키를 눌러 메인 세션으로 모읍니다.

사용법: dmux 도구 설치 후 사용

---

claude-devfleet

한 줄 요약: 여러 Claude 에이전트를 함대처럼 동시에 운영하는 스킬입니다.

비유: 해군 함대가 각 배에 임무를 배정하고 지휘관이 전체를 조율하듯이, 여러 Claude에게 각각 다른 코딩 임무를 줍니다.

언제 쓰나요? - 큰 프로젝트를 여러 Claude에게 동시에 시키고 싶을 때 - "인증 기능은 A가, 테스트는 B가, 문서는 C가" 식으로 분담할 때

어떻게 작동하나요? 작업 계획을 세우면 각 임무가 독립된 작업 공간에서 별도 Claude에 의해 실행됩니다. 하나가 끝나면 자동으로 결과를 합치고, 의존성이 있는 다음 임무를 자동 시작합니다. 최종 보고서로 무엇이 변경되었는지 확인할 수 있습니다.

사용법: Claude DevFleet MCP 서버 설정 후 사용

---

enterprise-agent-ops

한 줄 요약: 장시간 돌아가는 AI 에이전트를 안전하게 관리하는 운영 스킬입니다.

비유: 24시간 공장을 관리하는 것처럼, 계속 돌아가는 AI 시스템의 시작/중지/모니터링/보안을 관리합니다.

언제 쓰나요? - AI 에이전트를 서버에서 계속 돌릴 때 - 에이전트의 성능, 비용, 에러를 추적할 때 - 문제가 생기면 안전하게 중지시키고 복구할 때

어떻게 작동하나요? 에이전트의 전체 생명주기를 관리합니다. 시작, 일시정지, 중지, 재시작부터, 로그와 지표 모니터링, 권한 관리, 비상 중지 기능까지 포함됩니다. 문제가 발생하면 새 배포를 멈추고, 원인을 파악하고, 최소한의 변경으로 복구합니다.

사용법: 운영 환경 설정 시 참고

---

learned

한 줄 요약: continuous-learning 스킬이 자동으로 배운 내용을 저장하는 폴더입니다.

비유: 자동으로 채워지는 노트북과 같습니다. Claude가 배울 때마다 여기에 메모가 추가됩니다.

언제 쓰나요? - 자동으로 채워집니다 (SKILL.md 파일은 없음)

어떻게 작동하나요? continuous-learning이나 continuous-learning-v2가 세션에서 유용한 패턴을 발견하면 이 폴더에 저장합니다. 저장된 내용은 이후 세션에서 자동으로 참조됩니다.

사용법: 자동으로 실행됨

---

2. 내가 직접 쓰는 것 (학습/관리 도구)

직접 명령어를 입력하거나 요청해서 사용하는 스킬들입니다. 스마트폰에서 앱을 직접 터치해서 여는 것처럼요.

---

configure-ecc

한 줄 요약: ECC 스킬과 규칙을 선택해서 설치하는 마법사(설치 도우미)입니다.

비유: 스마트폰에서 앱스토어를 열고 원하는 앱을 골라 설치하는 것과 같습니다.

언제 쓰나요? - 처음 ECC를 설치할 때 - 새로운 스킬을 추가로 설치하고 싶을 때 - 기존 설치가 제대로 되었는지 확인할 때

어떻게 작동하나요? 단계별로 질문하면서 어떤 스킬을 설치할지 고릅니다. 선택한 스킬을 지정된 폴더에 복사하고, 제대로 설치됐는지 확인합니다. 필요하면 설치된 스킬을 프로젝트에 맞게 최적화할 수도 있습니다.

사용법: “configure ecc” 또는 “install ecc”라고 말하기

---

context-budget

한 줄 요약: Claude의 기억 공간을 얼마나 쓰고 있는지 분석해주는 스킬입니다.

비유: 스마트폰에서 “저장공간 관리”를 열어서 어떤 앱이 용량을 많이 차지하는지 보는 것과 같습니다.

언제 쓰나요? - Claude가 느려지거나 답변 품질이 떨어질 때 - 스킬이나 에이전트를 많이 추가한 후 - 새 스킬을 추가할 여유가 있는지 확인할 때

어떻게 작동하나요? 설치된 모든 스킬, 에이전트, MCP 서버, 규칙 파일을 스캔해서 각각 얼마나 기억 공간을 차지하는지 계산합니다. 너무 큰 파일, 중복되는 내용, 불필요한 구성요소를 찾아내서 "이것을 줄이면 공간이 확보됩니다"라고 알려줍니다.

사용법: `/context-budget`

---

skill-stocktake

한 줄 요약: 설치된 스킬들의 품질을 검사하는 스킬입니다.

비유: 가게 주인이 재고를 점검하면서 "이건 유통기한 지났네, 이건 상태 좋네" 확인하는 것과 같습니다.

언제 쓰나요? - 스킬이 많아져서 정리가 필요할 때 - 스킬의 품질을 점검하고 싶을 때 - 새 스킬을 추가한 후 전체 상태를 확인할 때

어떻게 작동하나요? 모든 스킬 파일을 품질 체크리스트로 평가합니다. "빠른 스캔" 모드는 최근 변경된 스킬만, "전체 점검" 모드는 모든 스킬을 검사합니다. 각 스킬에 점수를 매기고, 개선이 필요한 부분을 알려줍니다.

사용법: `/skill-stocktake` 또는 `/skill-stocktake full`

---

skill-comply

한 줄 요약: 스킬이나 규칙이 실제로 잘 지켜지고 있는지 테스트하는 스킬입니다.

비유: 학교에서 "교칙을 잘 지키는지" 불시 점검하는 것과 같습니다. 규칙을 정해놨지만 실제로 따르고 있는지 확인합니다.

언제 쓰나요? - 새 규칙을 추가한 후 제대로 적용되는지 확인할 때 - "이 스킬이 실제로 작동하긴 하는 건가?" 의문이 들 때 - 품질 관리 차원에서 정기적으로 점검할 때

어떻게 작동하나요? 스킬 파일을 읽어서 "이 스킬이 지켜지려면 이런 행동이 나와야 한다"는 기준을 자동 생성합니다. 그런 다음 실제로 Claude를 실행해서 그 행동이 나오는지 확인합니다. 쉬운 상황, 보통 상황, 어려운 상황 3단계로 테스트합니다.

사용법: `/skill-comply □□□□`

---

rules-distill

한 줄 요약: 여러 스킬에서 공통으로 나타나는 원칙을 뽑아서 규칙 파일로 만드는 스킬입니다.

비유: 여러 교과서에서 공통으로 나오는 핵심 내용만 뽑아서 "요약집"을 만드는 것과 같습니다.

언제 쓰나요? - 스킬을 많이 설치한 후 정기적으로 (월 1회 정도) - "비슷한 말이 여러 스킬에 있는 것 같은데..." 할 때 - 규칙이 부족하다고 느낄 때

어떻게 작동하나요? 모든 스킬과 규칙을 스캔해서 "보안은 항상 확인해라", "테스트를 먼저 작성해라" 같은 여러 스킬에 걸쳐 반복되는 원칙을 찾습니다. 이를 정리해서 기존 규칙 파일에 추가하거나 새 규칙 파일을 만듭니다.

사용법: `/rules-distill`

---

prompt-optimizer

한 줄 요약: Claude에게 요청하는 글(프롬프트)을 더 효과적으로 다듬어주는 스킬입니다.

비유: 글쓰기 코치가 "이렇게 쓰면 더 좋은 결과가 나올 거예요"라고 조언해주는 것과 같습니다.

언제 쓰나요? - "어떻게 물어봐야 Claude가 더 잘 이해할까?" 고민될 때 - 기존 프롬프트를 개선하고 싶을 때 - ECC 스킬을 활용한 최적의 프롬프트를 만들고 싶을 때

어떻게 작동하나요? 입력한 프롬프트를 분석해서 의도, 빠진 부분, 개선점을 파악합니다. 그리고 관련 ECC 스킬과 매칭해서 "이 스킬을 활용하면 더 좋은 결과가 나옵니다"라고 알려줍니다. 최적화된 프롬프트를 바로 복사해서 쓸 수 있게 출력합니다.

사용법: "optimize this prompt" 또는 `/prompt-optimize`

---

nanoclaw-repl

한 줄 요약: Claude와 대화를 세션 단위로 관리하는 가벼운 대화형 도구입니다.

비유: 메모장처럼 대화를 저장하고, 분류하고, 검색할 수 있는 Claude 전용 노트 앱입니다.

언제 쓰나요? - 여러 주제의 대화를 체계적으로 관리하고 싶을 때 - 이전 대화 내용을 검색하고 싶을 때 - 대화를 파일로 내보내고 싶을 때

어떻게 작동하나요? 대화를 마크다운 파일로 저장해서 세션별로 관리합니다. 모델 변경, 스킬 불러오기, 세션 분기, 검색, 압축, 내보내기 등의 기능을 슬래시 명령어로 제공합니다. 외부 의존성 없이 작동합니다.

사용법: `scripts/claw.js` 실행

---

team-builder

한 줄 요약: 여러 AI 에이전트 중에서 팀을 골라 구성하는 스킬입니다.

비유: 축구 감독이 선수 명단을 보고 "이번 경기에는 이 선수들로 팀을 짜자"고 결정하는 것과 같습니다.

언제 쓰나요? - 여러 에이전트 중 특정 작업에 맞는 팀을 구성할 때 - 어떤 에이전트가 있는지 둘러보고 싶을 때 - 보안 + 아키텍처 + SEO 같은 조합으로 팀을 만들 때

어떻게 작동하나요? 에이전트 파일들을 읽어서 이름, 설명, 도메인별로 정리한 메뉴를 보여줍니다. 원하는 에이전트를 선택하면 팀으로 구성됩니다. 폴더 구조 또는 파일 이름에서 자동으로 도메인을 추론합니다.

사용법: "team-builder" 실행

---

codebase-onboarding

한 줄 요약: 처음 보는 프로젝트를 빠르게 파악해서 가이드를 만들어주는 스킬입니다.

비유: 새 직장에 출근했을 때 선배가 "여기는 이런 구조고, 이 파일이 중요하고, 이런 규칙이 있어"라고 알려주는 것과 같습니다.

언제 쓰나요? - 새 프로젝트를 처음 열 때 - "이 프로젝트가 뭔지 전체적으로 파악하고 싶다"고 할 때 - CLAUDE.md 파일을 자동 생성하고 싶을 때

어떻게 작동하나요? 프로젝트의 설정 파일, 폴더 구조, 코드 패턴을 자동으로 스캔합니다. 어떤 기술을 쓰는지(React? Python?), 핵심 진입점이 어디인지, 어떤 컨벤션을 따르는지 분석합니다. 결과를 구조화된 온보딩 가이드로 만들어줍니다.

사용법: “help me understand this codebase” 또는 “onboard me”

---

documentation-lookup

한 줄 요약: 라이브러리나 프레임워크의 최신 문서를 실시간으로 찾아보는 스킬입니다.

비유: 사전이나 매뉴얼을 인터넷에서 찾아보는 것과 같습니다. Claude가 기억에 의존하지 않고 실제 최신 문서를 확인합니다.

언제 쓰나요? - “React에서 이거 어떻게 하지?” 같은 질문을 할 때 - 라이브러리 API 사용법을 정확히 알고 싶을 때 - 최신 버전의 사용법이 궁금할 때

어떻게 작동하나요? Context7이라는 문서 검색 도구를 사용합니다. 라이브러리 이름을 검색하면 최신 공식 문서에서 관련 내용을 가져옵니다. Claude의 학습 데이터가 아닌 실제 현재 문서를 참조하므로 더 정확합니다.

사용법: 자동으로 작동 (라이브러리 관련 질문 시)

---

iterative-retrieval

한 줄 요약: 필요한 정보를 단계적으로 찾아가는 검색 패턴입니다.

비유: 도서관에서 처음엔 목록을 보고, 그다음 관련 서가로 가고, 거기서 또 관련 책을 찾는 것처럼, 단계별로 점점 정확한 정보를 찾아갑니다.

언제 쓰나요? - 여러 에이전트가 코드를 탐색해야 할 때 - 어떤 파일이 필요한지 미리 알 수 없을 때 - “전부 보내면 너무 많고, 안 보내면 부족한” 상황에서

어떻게 작동하나요? 4단계 루프로 작동합니다: 보내기 > 평가 > 부족하면 더 찾기 > 다시 평가. 처음에는 대략적인 정보를 보내고, 에이전트가 더 필요한 것을 요청하면 추가로 찾아줍니다. 이를 반복하면서 필요한 맥락을 점진적으로 완성합니다.

사용법: 멀티 에이전트 워크플로우에서 자동 적용

---

3. 코딩할 때 도움주는 것

코드를 작성하고, 리뷰하고, 구조를 잡는 데 도움을 주는 스킬들입니다.

---

coding-standards

한 줄 요약: 코드를 깔끔하고 일관되게 작성하는 규칙 모음입니다.

비유: 글쓰기에 맞춤법과 문법 규칙이 있듯이, 코드에도 “이렇게 쓰면 좋다”는 표준이 있습니다.

언제 쓰나요? - 새 프로젝트를 시작할 때 - 코드 품질을 확인할 때 - 일관된 코드 스타일을 유지하고 싶을 때

어떻게 작동하나요? 읽기 좋은 코드 작성법(변수명은 명확하게, 함수는 동사-명사 형태 등), DRY(반복하지 마라), KISS(단순하게 만들어라) 같은 원칙을 담고 있습니다. TypeScript, JavaScript, React 프로젝트에 특화된 가이드를 제공합니다.

사용법: 자동으로 참조됨 (코딩 관련 작업 시)

---

api-design

한 줄 요약: 서버와 통신하는 규격(API)을 잘 설계하는 방법을 담은 스킬입니다.

비유: 식당의 메뉴판을 잘 만드는 것과 같습니다. 손님(사용자)이 무엇을 주문(요청)할 수 있고, 어떤 형태로 받을 수 있는지 깔끔하게 정리합니다.

언제 쓰나요? - 새로운 API를 설계할 때 - 기존 API를 개선할 때 - 에러 처리, 페이지 나누기, 버전 관리가 필요할 때

어떻게 작동하나요? URL 구조, 응답 형태, 에러 코드, 페이지 처리, 필터링 등 API를 일관되고 사용하기 쉽게 만드는 규칙을 제공합니다. 개발자들이 많이 사용하는 REST 방식의 표준 패턴을 기반으로 합니다.

사용법: 자동으로 참조됨 (API 설계 시)

---

backend-patterns

한 줄 요약: 서버 쪽 프로그램을 잘 만드는 설계 패턴 모음입니다.

비유: 건물의 배관, 전기 설계도처럼, 눈에 보이지 않지만 중요한 서버 구조를 잘 잡는 방법입니다.

언제 쓰나요? - 서버 코드를 작성할 때 - 데이터 흐름 구조를 설계할 때 - 캐싱, 에러 처리, 인증 등을 구현할 때

어떻게 작동하나요? 코드를 층(Layer)별로 나누는 방법, 데이터 요청 최적화, 캐시 전략, 비동기 처리, 미들웨어 구현 등 서버 개발에 필요한 핵심 패턴을 제공합니다.

사용법: 자동으로 참조됨 (서버 개발 시)

---

frontend-patterns

한 줄 요약: 사용자가 보는 화면(프론트엔드)을 잘 만드는 패턴 모음입니다.

비유: 인테리어 디자인 원칙처럼, 사용자가 직접 보고 만지는 부분을 깔끔하고 빠르게 만드는 방법입니다.

언제 쓰나요? - React 컴포넌트를 만들 때 - 상태 관리나 데이터 가져오기를 구현할 때 - 성능 최적화가 필요할 때

어떻게 작동하나요? React 컴포넌트 조합, 상태 관리(useState, Zustand 등), 데이터 불러오기 패턴, 성능 최적화(메모이제이션, 코드 분할), 폼 처리, 접근성 등 프론트엔드의 핵심 패턴을 제공합니다.

사용법: 자동으로 참조됨 (프론트엔드 개발 시)

---

react-components

한 줄 요약: Stitch 디자인 도구의 결과물을 React 코드로 변환하는 스킬입니다.

비유: 건축가의 도면을 보고 실제 건물을 짓는 것처럼, 디자인 파일을 보고 실제 동작하는 코드를 만듭니다.

언제 쓰나요? - Stitch에서 만든 디자인을 코드로 만들 때 - 디자인을 모듈형 컴포넌트로 분리할 때

어떻게 작동하나요? Stitch MCP에서 디자인 데이터를 가져와 스크린샷과 HTML을 다운로드합니다. 디자인을 분석해서 독립적인 React 컴포넌트 파일들로 나누고, 이벤트 처리 로직은 별도 hook으로 분리합니다.

사용법: Stitch MCP 연동 후 사용

---

database-migrations

한 줄 요약: 데이터 저장소(데이터베이스)의 구조를 안전하게 변경하는 방법입니다.

비유: 건물을 리모델링할 때 벽을 하나 옮기려면 설계도를 바꾸고, 안전하게 작업하고, 원래대로 돌릴 수 있는 계획이 필요한 것과 같습니다.

언제 쓰나요? - 데이터 저장 구조를 변경할 때 - 새 테이블이나 컬럼을 추가할 때 - 서비스 중단 없이 구조를 바꿔야 할 때

어떻게 작동하나요? 모든 데이터 구조 변경은 "마이그레이션"이라는 기록으로 관리합니다. 직접 데이터를 수정하는 것이 아니라, "이런 변경을 하겠다"는 계획서를 작성하고 실행합니다. 문제가 생기면 되돌릴 수 있도록 합니다.

사용법: 자동으로 참조됨 (DB 변경 시)

---

deployment-patterns

한 줄 요약: 코드를 실제 서비스 환경에 올리는(배포하는) 방법 모음입니다.

비유: 제품을 공장에서 만들어 매장에 진열하는 과정과 같습니다. 새 버전을 안전하게 갈아 끼우는 다양한 전략이 있습니다.

언제 쓰나요? - 코드를 서버에 배포할 때 - CI/CD 파이프라인을 설정할 때 - 배포 전략(블루-그린, 카나리 등)을 결정할 때

어떻게 작동하나요? 단계적 배포(하나씩 교체), 블루-그린 배포(새 환경 준비 후 한번에 전환), 카나리 배포(소수에게 먼저 공개) 등의 전략을 제공합니다. 건강 체크, 롤백 계획, 환경별 설정 방법도 포함됩니다.

사용법: 자동으로 참조됨 (배포 관련 작업 시)

---

docker-patterns

한 줄 요약: 프로그램을 독립된 상자(컨테이너)에 넣어서 어디서든 돌릴 수 있게 하는 방법입니다.

비유: 이삿짐을 박스에 정리해서 포장하면 어디로든 옮길 수 있듯이, 프로그램도 컨테이너에 담으면 어떤 서버에서든 동일하게 실행됩니다.

언제 쓰나요? - 개발 환경을 컨테이너로 만들 때 - 여러 서비스를 동시에 구성할 때 - 컨테이너 보안이나 네트워크 설정이 필요할 때

어떻게 작동하나요? Docker Compose로 여러 서비스(앱, 데이터베이스, 캐시)를 한꺼번에 설정하는 방법, Dockerfile을 효율적으로 작성하는 방법, 네트워크와 저장소 설정 방법을 제공합니다.

사용법: 자동으로 참조됨 (Docker 관련 작업 시)

---

postgres-patterns

한 줄 요약: PostgreSQL 데이터베이스를 잘 사용하는 방법 모음입니다.

비유: 엑셀을 쓸 때 "이렇게 정리하면 검색이 빠르다"는 노하우와 같습니다.

언제 쓰나요? - 데이터베이스 구조를 설계할 때 - 느린 검색을 빠르게 만들 때 - 보안(Row Level Security) 설정이 필요할 때

어떻게 작동하나요? 데이터 조회 패턴별 최적의 인덱스 유형, 스키마 설계 원칙, 연결 풀링 설정 등을 제공합니다. 특히 검색 속도를 높이는 인덱스 설정 가이드가 핵심입니다.

사용법: 자동으로 참조됨 (PostgreSQL 작업 시)

---

content-hash-cache-pattern

한 줄 요약: 파일 처리 결과를 캐시(임시 저장)해서 같은 파일을 다시 처리하지 않는 패턴입니다.

비유: 한번 번역한 문서에 "번역 완료" 도장을 찍어놓고, 다음에 같은 문서가 오면 이미 번역된 결과를 바로 보여주는 것과 같습니다.

언제 쓰나요? - 같은 파일을 반복 처리하는 작업이 있을 때 - PDF 변환, 텍스트 추출 같은 시간이 오래 걸리는 작업에서 - 파일이 변경되면 자동으로 다시 처리해야 할 때

어떻게 작동하나요? 파일 내용의 고유 지문(SHA-256 해시)을 캐시 키로 사용합니다. 파일 이름이 바뀌어도 내용이 같으면 캐시를 재사용하고, 내용이 바뀌면 자동으로 무효화됩니다.

사용법: 파일 처리 파이프라인에서 적용

---

agentic-engineering

한 줄 요약: AI 에이전트가 대부분의 코딩을 하고 사람은 품질을 관리하는 방식입니다.

비유: 사장이 직원에게 일을 맡기고, 핵심적인 검수만 직접 하는 것과 같습니다.

언제 쓰나요? - AI에게 코딩을 맡기는 워크플로우를 설정할 때 - 작업을 작은 단위로 나눠서 AI에게 할당할 때 - 비용을 관리하면서 AI를 효율적으로 쓰고 싶을 때

어떻게 작동하나요? 작업을 15분 단위로 분해하고, 복잡도에 따라 적합한 AI 모델(저렴한 것 vs 고급)을 배정합니다. "평가 > 구현 > 재평가" 루프로 품질을 보장합니다. 코드 리뷰에서는 스타일 대신 보안과 에러 처리에 집중합니다.

사용법: 작업 계획 시 참고

---

ai-first-engineering

한 줄 요약: AI가 코드의 많은 부분을 생성하는 팀을 위한 운영 방식입니다.

비유: 로봇이 조립을 하고 사람이 검수하는 공장의 운영 매뉴얼과 같습니다.

언제 쓰나요? - 팀 전체가 AI를 활용해서 코딩할 때 - 프로세스, 리뷰, 아키텍처를 AI 중심으로 재설계할 때

어떻게 작동하나요? AI가 생성한 코드의 리뷰 방식, 품질 검증 절차, 팀 역할 분담 등을 정의합니다. AI 코드의 일반적인 취약점(동일 모델이 작성하고 리뷰하면 같은 실수를 놓침)을 보완하는 전략을 제공합니다.

사용법: 팀 프로세스 설계 시 참고

---

agent-eval

한 줄 요약: 여러 AI 코딩 도구의 성능을 비교 테스트하는 스킬입니다.

비유: 자동차를 살 때 여러 차종을 시승하고 비교하는 것처럼, Claude Code, Aider, Codex 등을 같은 과제로 테스트해서 비교합니다.

언제 쓰나요? - 어떤 AI 코딩 도구가 우리 프로젝트에 맞는지 비교할 때 - 새 AI 모델이 나왔을 때 기존 대비 성능을 확인할 때 - 데이터 기반으로 도구 선택을 결정할 때

어떻게 작동하나요? 같은 과제를 여러 AI 에이전트에게 주고, 성공률, 비용, 시간, 일관성을 측정합니다. 각 에이전트는 독립된 환경에서 실행되어 서로 영향을 주지 않습니다.

사용법: `agent-eval run --task □□ --agent claude-code --agent aider`

---

agent-harness-construction

한 줄 요약: AI 에이전트가 도구를 더 잘 쓸 수 있도록 도구 정의와 응답 형태를 최적화하는 스킬입니다.

비유: 조종사에게 맞게 조종석을 설계하는 것처럼, AI에게 맞게 도구와 인터페이스를 설계합니다.

언제 쓰나요? - AI 에이전트가 도구를 잘 못 쓰거나 에러가 많을 때 - 에이전트의 작업 완료율을 높이고 싶을 때 - 새로운 도구를 에이전트에게 제공할 때

어떻게 작동하나요? 도구 이름을 명확하게, 입력/출력을 일관되게, 에러 발생 시 복구 방법을 명시하는 등의 설계 원칙을 제공합니다. 도구 응답에는 항상 상태, 요약, 다음 행동, 결과물 정보를 포함하도록 합니다.

사용법: 에이전트 시스템 설계 시 참고

---

blueprint

한 줄 요약: 큰 프로젝트를 단계별 실행 계획으로 분해하는 스킬입니다.

비유: 건축가가 건물을 짓기 전에 설계도와 공정표를 만드는 것과 같습니다. "1단계: 기초 공사, 2단계: 골조..." 식으로 나눕니다.

언제 쓰나요? - 여러 세션에 걸친 큰 작업을 계획할 때 - "이걸 어디서부터 시작하지?" 할 때 - 병렬로 작업할 부분과 순서가 있는 부분을 나눌 때

어떻게 작동하나요? 한 줄짜리 목표를 입력하면, 의존 관계 그래프, 병렬 가능 단계, 위험 요소 분석을 포함한 상세 실행 계획을 만듭니다. 각 단계는 새 Claude가 읽어도 바로 실행할 수 있도록 독립적인 맥락을 포함합니다.

사용법: "plan this" 또는 "blueprint for..."

---

ralphinho-rfc-pipeline

한 줄 요약: 큰 기능을 설계서(RFC) 기반으로 나눠서 여러 에이전트가 순서대로 처리하는 파이프라인입니다.

비유: 대규모 공사에서 설계서를 먼저 만들고, 공정별로 팀을 배정하고, 각 팀이 완료하면 다음 팀이 시작하는 방식입니다.

언제 쓰나요? - 한 번에 처리하기 너무 큰 기능을 개발할 때 - 독립적으로 검증 가능한 단위로 작업을 나눠야 할 때

어떻게 작동하나요? RFC 수령 > 의존 관계 분석 > 작업 단위 분배 > 구현 > 검증 > 통합 > 최종 확인 순으로 진행됩니다. 각 작업은 복잡도 등급(1~3)으로 분류되어 적절한 수준의 검증을 받습니다.

사용법: 대규모 기능 개발 시 적용

---

architecture-decision-records

한 줄 요약: "왜 이 기술을 선택했는지" 결정 기록을 남기는 스킬입니다.

비유: 회의록을 남기는 것과 같습니다. "PostgreSQL을 선택한 이유", "REST API를 고른 이유" 등을 기록해서 나중에 "왜 이렇게 했지?" 할 때 찾아볼 수 있게 합니다.

언제 쓰나요? - 중요한 기술 선택을 할 때 (프레임워크, 데이터베이스 등) - "왜 이걸 선택했지?"라고 궁금할 때 기존 기록을 찾을 때 - "이 결정을 기록해두자"고 할 때

어떻게 작동하나요? 대화 중 기술 선택 순간을 감지하면, 맥락, 결정 내용, 고려했던 대안, 장단점을 구조화된 문서(ADR)로 작성합니다. docs/adr/ 폴더에 저장되어 누구든 찾아볼 수 있습니다.

사용법: "ADR this" 또는 자동 감지 후 제안

---

project-guidelines-example

한 줄 요약: 프로젝트별 스킬을 만들 때 참고할 수 있는 템플릿입니다.

비유: "공문서 양식"처럼, 자기 프로젝트 스킬을 만들 때 이 형식을 참고하면 됩니다.

언제 쓰나요? - 우리 프로젝트만의 가이드 스킬을 만들 때 - 프로젝트 구조, 코드 패턴, 테스트 방법을 정리할 때

어떻게 작동하나요? 실제 프로덕션 앱을 기반으로 한 예시 템플릿입니다. 아키텍처 개요, 파일 구조, 코드 패턴, 테스트 요구사항, 배포 워크플로우 등이 포함되어 있어서 이걸 참고해서 자기 프로젝트 가이드를 작성하면 됩니다.

사용법: 새 프로젝트 설정 시 참고

---

search-first

한 줄 요약: 코드를 직접 작성하기 전에 이미 만들어진 도구가 있는지 먼저 찾아보는 스킬입니다.

비유: 직접 요리하기 전에 "배달 가능한 곳 있나?" 먼저 확인하는 것과 같습니다. 이미 있는 것을 다시 만들 필요는 없으니까요.

언제 쓰나요? - 새 기능을 시작할 때 ("혹시 이미 있는 라이브러리 없나?") - 새로운 도구나 연동을 추가할 때 - 바퀴를 다시 발명하고 싶지 않을 때

어떻게 작동하나요? 필요한 기능 분석 > npm/PyPI 등 패키지 저장소 검색 > 기존 MCP/스킬 검색 > GitHub 검색을 병렬로 실행합니다. 후보를 평가(기능, 유지보수, 커뮤니티, 라이선스)한 후 "있는 걸 쓸지, 수정해서 쓸지, 직접 만들지" 결정합니다.

사용법: `/search-first` 또는 자동으로 활성화

---

rapid-prototyping

한 줄 요약: 빠르게 시제품을 만들어보는 "바이브 코딩" 스킬입니다.

비유: 도자기를 굽기 전에 찰흙으로 대충 형태를 잡아보는 것과 같습니다. 완벽하지 않아도 되니까 빨리 만들어보고, 괜찮으면 제대로 만듭니다.

언제 쓰나요? - 아이디어를 빠르게 검증하고 싶을 때 - 이해관계자에게 데모를 보여줘야 할 때 - "일단 만들어보고 판단하자"는 단계에서

어떻게 작동하나요? AI에게 "대충 만들어봐"라고 하면 빠르게 동작하는 프로토타입을 만듭니다. 코드 품질보다 학습 속도가 중요한 탐색 단계에서 사용합니다. 결과가 괜찮으면 그때 제대로 다시 만듭니다.

사용법: 탐색/프로토타이핑 단계에서 사용

---

cost-aware-llm-pipeline

한 줄 요약: AI API 비용을 절약하면서 품질을 유지하는 방법입니다.

비유: 간단한 질문은 인턴에게, 어려운 질문은 전문가에게 물어보는 것처럼, 쉬운 작업은 저렴한 모델에게, 어려운 작업은 고급 모델에게 맡깁니다.

언제 쓰나요? - AI API 비용이 걱정될 때 - 많은 작업을 자동 처리해야 할 때 - 예산 한도 내에서 AI를 사용해야 할 때

어떻게 작동하나요? 작업 복잡도에 따라 자동으로 모델을 선택합니다(짧은 텍스트 = 저렴한 모델, 긴 텍스트 = 고급 모델). 예산 추적, 재시도 로직, 프롬프트 캐싱을 조합해서 비용을 관리합니다.

사용법: AI 파이프라인 설계 시 적용

---

claude-api

한 줄 요약: Claude API를 활용해서 앱을 만드는 방법 모음입니다.

비유: 다른 프로그램에서 Claude를 "전화"로 부르는 방법입니다. 메시지 보내기, 도구 사용, 이미지 분석 등의 사용법을 담고 있습니다.

언제 쓰나요? - Claude API를 사용하는 앱을 만들 때 - 도구 사용, 스트리밍, 이미지 분석을 구현할 때 - 에이전트 워크플로우를 만들 때

어떻게 작동하나요? Python과 TypeScript SDK 사용법, 모델 선택 가이드(Opus=복잡, Sonnet=일반, Haiku=빠른), 메시지 전송, 스트리밍, 도구 정의, 이미지 분석, 배치 처리, 프롬프트 캐싱 등의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Claude API 사용 시)

---

mcp-server-patterns

한 줄 요약: MCP(Model Context Protocol) 서버를 만드는 방법 모음입니다.

비유: AI 비서에게 새로운 "기능 플러그인"을 만들어서 연결하는 방법입니다.

언제 쓰나요? - AI가 사용할 수 있는 새로운 도구를 만들 때 - MCP 서버를 구현하거나 디버그할 때

어떻게 작동하나요? 도구(Tools), 리소스(Resources), 프롬프트(Prompts)를 정의하고 등록하는 방법을 제공합니다. 로컬 연결(stdio)과 원격 연결(HTTP) 방식을 모두 지원합니다.

사용법: 자동으로 참조됨 (MCP 서버 개발 시)

---

regex-vs-llm-structured-text

한 줄 요약: 텍스트를 분석할 때 정규표현식(패턴 매칭)과 AI 중 어떤 것을 쓸지 결정하는 스킬입니다.

비유: 계산기로 충분한 계산을 굳이 슈퍼컴퓨터로 할 필요 없듯이, 간단한 텍스트 처리는 패턴 매칭으로, 복잡한 것만 AI로 처리합니다.

언제 쓰나요? - 반복되는 형식의 텍스트를 자동 분석할 때 - 비용/정확도 균형을 맞추고 싶을 때

어떻게 작동하나요? 패턴이 일정한 텍스트(95%+)는 정규표현식으로 빠르고 저렴하게 처리하고, 나머지 불규칙한 부분만 AI를 사용합니다. 이렇게 하면 비용을 크게 줄이면서도 정확도를 유지할 수 있습니다.

사용법: 텍스트 파이프라인 설계 시 참고

---

4. 테스트/품질 관리

코드가 제대로 작동하는지 확인하고, 품질을 보장하는 스킬들입니다.

---

tdd-workflow

한 줄 요약: "테스트를 먼저 만들고, 그다음 코드를 작성하는" 개발 방법입니다.

비유: 시험 문제를 먼저 만들고, 그 문제를 풀 수 있는 교재를 만드는 것과 같습니다. 정답 기준이 먼저이므로 더 확실한 코드가 나옵니다.

언제 쓰나요? - 새 기능을 만들 때 - 버그를 고칠 때 - 코드를 정리(리팩토링)할 때

어떻게 작동하나요? 1. 먼저 "이 기능은 이래야 한다"는 테스트를 작성합니다 (아직 실패함). 2. 테스트를 통과시키는 최소한의 코드를 작성합니다. 3. 코드를 더 깔끔하게 정리합니다. 최소 80% 이상의 코드가 테스트로 검증되어야 합니다.

사용법: 자동으로 참조됨 (코드 작성 시)

---

verification-loop

한 줄 요약: 코드 변경 후 빌드, 타입 체크, 린트, 테스트를 순서대로 돌리는 검증 과정입니다.

비유: 비행기 이륙 전 체크리스트처럼, 코드를 배포하기 전에 모든 항목을 하나하나 확인합니다.

언제 쓰나요? - 기능 완성 후 최종 확인할 때 - Pull Request를 만들기 전에 - 리팩토링 후 아무것도 안 깨졌는지 확인할 때

어떻게 작동하나요? 빌드 > 타입 체크 > 린트(코드 스타일 검사) > 테스트 > 보안 검사 순서로 진행됩니다. 앞 단계가 실패하면 멈추고 고친 후 다시 시작합니다. 모든 단계를 통과해야 "완료"입니다.

사용법: 자동으로 참조됨 (작업 완료 후)

---

e2e-testing

한 줄 요약: 실제 사용자처럼 화면을 클릭하고 입력하면서 테스트하는 방법입니다.

비유: 미스터리 쇼퍼가 실제 고객처럼 매장을 이용하면서 서비스를 점검하는 것과 같습니다.

언제 쓰나요? - 로그인 > 기능 사용 > 로그아웃 같은 전체 흐름을 테스트할 때 - 화면이 실제로 잘 동작하는지 확인할 때 - CI/CD 파이프라인에 자동 테스트를 추가할 때

어떻게 작동하나요? Playwright라는 도구를 사용해서 브라우저를 자동으로 조작합니다. 버튼 클릭, 텍스트 입력, 페이지 이동 등을 사람 대신 자동으로 수행하고, 결과가 예상대로인지 확인합니다.

사용법: 자동으로 참조됨 (E2E 테스트 작성 시)

---

ai-regression-testing

한 줄 요약: AI가 작성한 코드에서 놓치기 쉬운 버그를 잡는 전용 테스트 방법입니다.

비유: 같은 사람이 글 쓰고 교정하면 오타를 못 찾듯이, AI도 자기가 쓴 코드를 자기가 리뷰하면 같은 실수를 놓칩니다. 이 스킬은 그런 "사각지대"를 잡습니다.

언제 쓰나요? - AI가 코드를 수정한 후 - 같은 버그가 계속 재발할 때 - AI의 "자기 검토"가 신뢰가 안 될 때

어떻게 작동하나요? AI가 코드를 작성하면 자동화된 테스트를 즉시 실행합니다. AI의 리뷰에 의존하지 않고, 실제로 코드를 돌려서 결과를 확인합니다. 특히 샌드박스 모드와 실제 모드 같은 다중 경로를 모두 검사합니다.

사용법: `/bug-check` 또는 자동으로 활성화

---

benchmark

한 줄 요약: 웹사이트나 API의 속도와 성능을 측정하는 스킬입니다.

비유: 자동차의 연비와 가속 성능을 측정하는 것처럼, 웹사이트의 로딩 속도와 API 응답 시간을 측정합니다.

언제 쓰나요? - 성능 기준치를 설정할 때 - "느려진 것 같은데" 실제로 확인할 때 - 코드 변경 전후 성능 비교할 때

어떻게 작동하나요? 페이지 로딩 속도(LCP, CLS 등 Core Web Vitals), API 응답 시간, 번들 크기 등을 자동으로 측정합니다. 목표치(예: 로딩 2.5초 이내)와 비교해서 통과/미달을 판정합니다.

사용법: `/benchmark` □□□□□ 또는 자동 활성화

---

eval-harness

한 줄 요약: Claude의 작업 완료 기준을 정의하고 자동으로 평가하는 프레임워크입니다.

비유: 시험 문제와 채점 기준표를 만들어놓고, Claude가 작업을 끝내면 자동으로 채점하는 시스템입니다.

언제 쓰나요? - "Claude가 이 작업을 정말 잘 해냈는지" 객관적으로 평가할 때 - 모델 버전이 바뀔 때 성능을 비교할 때 - 에이전트의 신뢰도를 측정할 때

어떻게 작동하나요? 작업 전에 성공 기준을 정의합니다. Claude가 작업을 끝내면 그 기준에 맞는지 자동으로 평가합니다. pass@k(k번 중 몇 번 성공하나) 같은 지표로 신뢰도를 측정합니다.

사용법: 평가 기반 개발(EDD) 워크플로우에서 사용

---

plankton-code-quality

한 줄 요약: 코드를 수정할 때마다 자동으로 스타일 검사와 수정을 해주는 스킬입니다.

비유: 맞춤법 검사기가 글을 쓸 때마다 자동으로 틀린 부분을 고쳐주는 것과 같습니다.

언제 쓰나요? - 파일을 수정할 때마다 자동으로 실행됩니다 - 코드 스타일을 자동으로 통일하고 싶을 때

어떻게 작동하나요? 3단계로 작동합니다. 1단계: 포매터가 40~50%를 자동 수정합니다. 2단계: 고칠 수 없는 문제를 수집합니다. 3단계: 문제의 복잡도에 따라 적합한 AI 모델을 불러서 수정합니다.

사용법: 자동으로 실행됨 (PostToolUse hook)

---

browser-qa

한 줄 요약: 실제 브라우저에서 웹사이트를 자동으로 테스트하는 스킬입니다.

비유: 품질 관리팀이 실제로 사이트를 돌아다니며 "여기 깨졌네, 여기 느리네" 확인하는 것을 자동화한 것입니다.

언제 쓰나요? - 기능 배포 후 실제로 잘 되는지 확인할 때 - 레이아웃, 폼, 링크가 제대로 동작하는지 테스트할 때 - 접근성과 반응형 디자인을 검사할 때

어떻게 작동하나요? 브라우저 자동화 도구로 실제 페이지를 열어서 3단계로 테스트합니다. 1. 스모크 테스트: 페이지가 정상인지 기본 확인. 2. 인터랙션 테스트: 링크, 폼, 인증 흐름 테스트. 3. 비주얼 회귀: 여러 화면 크기에서 스크린샷 비교.

사용법: `/browser-qa □□□□□`

---

click-path-audit

한 줄 요약: 버튼을 눌렀을 때 실제로 의도한 결과가 나오는지 추적하는 스킬입니다.

비유: "새 이메일" 버튼을 눌렀는데 실제로 아무 일도 안 일어나는 경우를 찾아냅니다. 개별 기능은 다 동작하는데, 조합하면 서로 취소해서 결과가 안 나오는 버그를 잡습니다.

언제 쓰나요? - 일반 디버깅으로 못 찾는 버그가 있을 때 - 대규모 코드 변경 후 버튼들이 제대로 작동하는지 확인할 때 - 사용자가 "이 버튼이 안 돼요"라고 할 때

어떻게 작동하나요? 모든 클릭 가능한 요소에 대해: 클릭 핸들러 확인 > 호출되는 함수를 순서대로 추적 > 각 함수가 어떤 상태를 읽고/쓰는지 확인 > 뒤의 함수가 앞의 결과를 되돌리는지 확인 > 최종 상태가 버튼 레이블과 일치하는지 확인합니다.

사용법: `/click-path-audit`

---

santa-method

한 줄 요약: 두 명의 독립 검토자가 모두 "합격"해야 결과물을 통과시키는 검증 방법입니다.

비유: 산타클로스가 "리스트를 만들고, 두 번 확인한다(Making a list, checking it twice)"는 것처럼, AI가 만든 결과물을 두 명의 독립 검토자가 각각 확인합니다.

언제 쓰나요? - 프로덕션에 배포되는 코드에 - 정확성이 중요한 기술 문서나 콘텐츠에 - 대량 생성 시 AI 환각(없는 사실 생성)을 방지할 때

어떻게 작동하나요? 1단계: 생성자(Agent A)가 결과물을 만듭니다. 2단계: 검토자 B와 C가 서로 모르는 상태에서 같은 기준으로 각각 검토합니다. 둘 다 합격하면 통과, 하나라도 실패하면 수정 후 재검토합니다.

사용법: 품질이 중요한 결과물 생성 시 적용

---

kdh-playwright-e2e-full-auto-24-7-tmux

한 줄 요약: CORTHEX 프로젝트 전용 자동 E2E 테스트 + 버그 수정 + 배포 파이프라인 (TMUX 버전)입니다.

비유: 24시간 돌아가는 자동 품질 검사 로봇이 사이트를 계속 테스트하고, 문제를 발견하면 자동으로 수정하고 배포합니다.

언제 쓰나요? - Oracle VPS에서 tmux로 24시간 자동 테스트를 돌릴 때 - 4개 팀 에이전트가 병렬로 테스트할 때

어떻게 작동하나요? 30분마다 자동 사이클이 돌아갑니다. 사이트 접속 > API 테스트 > 화면 테스트 > 버그 발견 시 자동 수정 > 배포 > 보고서 작성. 4개 에이전트가 동시에 작업해서 TMUX 환경에서 3~4배 빠릅니다.

사용법: `/loop 30m /kdh-playwright-e2e-full-auto-24-7-tmux`

---

kdh-playwright-e2e-full-auto-24-7-vs

한 줄 요약: 위와 같은 자동 테스트 파이프라인의 VS Code 버전입니다.

비유: 같은 자동 검사 로봇인데, VS Code 안에서 돌아가는 버전입니다.

언제 쓰나요? - VS Code Claude Code에서 15분 간격 자동 테스트를 돌릴 때 - 단발 E2E 테스트를 실행할 때

어떻게 작동하나요? 15분마다: 사이트 살아있나 확인 > 로그인 > API 스모크 테스트 > 페이지별 테스트 > 버그 발견 시 자동 수정 > 배포 > 정리 > 보고서. admin, ceo, api-only, quick 등 범위를 지정할 수 있습니다.

사용법: `/loop 15m /kdh-playwright-e2e-full-auto-24-7`

---

5. 보안

코드와 시스템의 보안을 검사하고 강화하는 스킬들입니다.

---

security-review

한 줄 요약: 코드의 보안 취약점을 체크리스트로 검사하는 스킬입니다.

비유: 건물의 방범 시스템을 점검하는 것처럼, 코드에 "잠금장치가 빠진 문"이 없는지 확인합니다.

언제 쓰나요? - 로그인/인증 기능을 만들 때 - 사용자 입력을 처리할 때 - API를 만들 때 - 비밀번호나 API 키를 다룰 때

어떻게 작동하나요? 비밀 정보 관리(코드에 직접 쓰지 않기), 입력 검증(위험한 입력 차단), 인증/인가(허가된 사람만 접근), 파일 업로드 보안, HTTPS 설정 등을 체크합니다. 잘못된 예시와 올바른 예시를 함께 보여줍니다.

사용법: 자동으로 참조됨 (보안 관련 코드 작성 시)

---

security-scan

한 줄 요약: Claude Code 설정 자체의 보안을 검사하는 스킬입니다.

비유: 사무실의 보안 카메라 시스템 자체가 안전한지 점검하는 것과 같습니다.

언제 쓰나요? - 새 프로젝트를 설정할 때 - Claude Code 설정을 변경한 후 - 보안 점검이 필요할 때

어떻게 작동하나요? AgentShield를 사용해서 CLAUDE.md(하드코딩된 비밀, 프롬프트 인젝션), settings.json(너무 넓은 권한), MCP 설정(위험한 서버), hooks(명령어 인젝션), 에이전트 정의(무제한 도구 접근) 등을 검사합니다.

사용법: `/security-scan`

---

safety-guard

한 줄 요약: 위험한 명령어를 실행하기 전에 차단하고 경고하는 스킬입니다.

비유: 자동차의 긴급 제동 시스템처럼, "rm -rf /"같은 위험한 명령어를 감지하면 자동으로 멈추고 경고합니다.

언제 쓰나요? - 프로덕션 환경에서 작업할 때 - 에이전트가 자율적으로 실행될 때 - 특정 폴더만 수정 가능하도록 제한하고 싶을 때

어떻게 작동하나요? 3가지 모드가 있습니다. "조심 모드": 위험한 명령어(rm -rf, git push –force, DROP TABLE 등) 감지 시 경고 + 대안 제시. "동결 모드": 특정 폴더만 수정 가능하도록 제한. "감사 모드": 모든 파일 변경 기록.

사용법: `/safety-guard` 또는 자동 활성화

---

6. 디자인/UI

사용자 인터페이스와 디자인 관련 스킬들입니다.

---

ui-ux-pro-max

한 줄 요약: 50가지 디자인 스타일, 97가지 색상 팔레트, 57가지 글꼴 조합을 담은 종합 디자인 가이드입니다.

비유: 인테리어 디자이너의 종합 자료집처럼, 색상, 글꼴, 레이아웃, 애니메이션 등 디자인에 필요한 모든 참고자료가 들어있습니다.

언제 쓰나요? - UI 컴포넌트나 페이지를 디자인할 때 - 색상과 글꼴을 고를 때 - 접근성 요구사항을 확인할 때

어떻게 작동하나요? 접근성(대비 4.5:1 이상), 터치 인터랙션, 성능, 레이아웃, 타이포그래피, 애니메이션, 스타일 선택, 차트 디자인을 우선순위 별로 가이드합니다. 9개 기술 스택(React, Next.js, Vue, Svelte, SwiftUI 등)별 구현 방법도 포함됩니다.

사용법: 자동으로 참조됨 (UI 작업 시)

---

design-system

한 줄 요약: 프로젝트의 디자인 규칙(색상, 글꼴, 간격 등)을 자동 생성하거나 검사하는 스킬입니다.

비유: 패션 브랜드의 "스타일 가이드"를 만드는 것과 같습니다. "우리 브랜드의 주요 색은 이거, 글꼴은 저거, 버튼 모양은 이거"라고 정합니다.

언제 쓰나요? - 새 프로젝트의 디자인 시스템을 만들 때 - 기존 UI의 일관성을 검사할 때 - UI가 "뭔가 이상한데" 정확히 뭐가 문제인지 모를 때

어떻게 작동하나요? "생성 모드": 코드를 스캔해서 사용 중인 색상/글꼴/간격을 추출하고, 경쟁사를 참고해서 디자인 토큰을 제안합니다. "감사 모드": 10가지 항목(색상 일관성, 타이포그래피, 간격 등)에 각각 0~10점을 매깁니다.

사용법: `/design-system` 또는 자동 활성화

---

design-md

한 줄 요약: Stitch 디자인 도구의 프로젝트를 분석해서 디자인 시스템 문서를 만드는 스킬입니다.

비유: 이미 만들어진 디자인을 보고 "이 디자인의 규칙은 뭐지?"를 역으로 분석해서 설명서를 만드는 것입니다.

언제 쓰나요? - Stitch 프로젝트의 디자인 언어를 문서화할 때 - 새 화면을 기존 디자인과 일관되게 만들어야 할 때

어떻게 작동하나요? Stitch MCP에서 화면 데이터와 스크린샷을 가져와 분석합니다. 색상, 글꼴, 간격, 구성요소 스타일 등을 추출해서 DESIGN.md 파일로 정리합니다. 이 문서를 기반으로 새 화면을 만들면 기존과 일관된 디자인이 나옵니다.

사용법: Stitch MCP 연동 후 사용

---

design-principles

한 줄 요약: 시각 디자인의 근본 원리(시각적 위계, 구성, 색상 이론 등)를 담은 스킬입니다.

비유: 미술 수업에서 배우는 "구도, 색채, 원근법" 같은 기본기입니다. 이 원칙은 의견이 아니라 인간의 시각 인지 방식에 기반한 법칙입니다.

언제 쓰나요? - 디자인 결정을 할 때 ("이게 왜 보기 좋은 거지?") - 디자인이 "뭔가 이상한데" 원인을 찾을 때 - 디자인 기본기를 참고할 때

어떻게 작동하나요? 시각적 위계(큰 것 = 중요), 게슈탈트 심리학(유사한 것은 묶여 보임), 구성과 균형, 색상 이론, 타이포그래피 원리를 체계적으로 정리합니다. 디자인을 흐릿하게 보았을 때 포인트가 보이는지로 위계를 테스트하는 방법 등도 포함됩니다.

사용법: 자동으로 참조됨 (디자인 관련 작업 시)

---

design-movements

한 줄 요약: 역사적 디자인 운동(바우하우스, 아르데코, 미니멀리즘 등)의 계보와 특성을 담은 스킬입니다.

비유: 패션 역사를 알면 현재 트렌드가 이해되듯이, 디자인 역사를 알면 왜 이 스타일이 이런 느낌인지 이해할 수 있습니다.

언제 쓰나요? - 프로젝트의 미적 방향을 정할 때 - 특정 스타일의 문화적 맥락을 이해하고 싶을 때 - 트렌드 사이클을 예측할 때

어떻게 작동하나요? 아르누보, 바우하우스, 스위스 국제주의, 포스트모더니즘, 멤피스 등 주요 디자인 운동의 역사, 특성, 영향을 계보도와 함께 정리합니다. 각 운동이 어떤 반작용으로 탄생했는지 이해하면 다음 트렌드를 예측할 수 있습니다.

사용법: 자동으로 참조됨 (디자인 방향 결정 시)

---

design-masters

한 줄 요약: 전설적인 디자이너들(솔 바스, 디터 람스 등)의 원칙과 작품을 담은 스킬입니다.

비유: 명장(名匠)들의 기술과 철학을 배우는 도제 교육과 같습니다.

언제 쓰나요? - 깊이 있는 디자인 영감이 필요할 때 - "왜 이 디자인이 명작인지" 이해하고 싶을 때 - 프로젝트에 적합한 디자인 방향의 레퍼런스를 찾을 때

어떻게 작동하나요? 솔 바스(기하학적 단순화), 디터 람스(10가지 좋은 디자인 원칙), 마시모 비넬리(그리드 시스템), 폴라 셰어(대담한 타이포그래피) 등 전설적 디자이너들의 철학, 대표작, 배울 점을 정리합니다.

사용법: 자동으로 참조됨 (디자인 영감 필요 시)

---

brand-systems

한 줄 요약: 브랜드 아이덴티티(로고, 색상, 목소리 등)를 체계적으로 만드는 스킬입니다.

비유: 기업의 CI(Corporate Identity) 매뉴얼을 만드는 것과 같습니다. 로고 위치, 색상 코드, 어투까지 모두 통일합니다.

언제 쓰나요? - 새 브랜드를 만들 때 - 브랜드를 리뉴얼할 때 - 브랜드 가이드라인 문서를 작성할 때

어떻게 작동하나요? 전략 계층(포지셔닝, 미션, 비전) > 시각 계층(로고, 색상, 글꼴) > 목소리 계층(어투, 톤) > 적용 규칙(어디에 어떻게 쓰는지)을 순서대로 정의합니다.

사용법: 브랜드 작업 시 참고

---

premium-saas-design

한 줄 요약: 고급스러운 SaaS 웹사이트를 만드는 전문 디자인 프레임워크입니다.

비유: 인테리어 디자이너가 고급 매장을 설계하듯이, "이 사이트는 비싸 보인다"는 느낌을 주는 웹사이트를 만드는 방법입니다.

언제 쓰나요? - 전문적이고 신뢰감 있는 SaaS 웹사이트를 만들 때 - AI와 함께 디자인 퀄리티를 높이고 싶을 때

어떻게 작동하나요? "정의 > 제작 > 검토 > 개선" 루프를 반복합니다. 7가지 맥락 문서(프로젝트 브리프, 디자인 시스템 등)를 먼저 만들어서 AI에게 명확한 방향을 제시합니다.

사용법: SaaS 디자인 프로젝트에서 적용

---

liquid-glass-design

한 줄 요약: Apple iOS 26의 새로운 "Liquid Glass" 디자인 시스템을 구현하는 스킬입니다.

비유: 유리 소재처럼 뒤가 비치고, 빛이 반사되고, 터치하면 반응하는 새로운 Apple 디자인 스타일입니다.

언제 쓰나요? - iOS 26+ 앱을 만들거나 업데이트할 때 - 유리 효과의 버튼, 카드, 툴바를 구현할 때 - 기존 블러 효과를 새 Liquid Glass API로 마이그레이션할 때

어떻게 작동하나요? SwiftUI의 .glassEffect() 수정자를 사용해서 투명한 유리 질감을 구현합니다. 형태, 색조, 인터랙션 애니메이션, 위젯 적용 방법 등을 제공합니다.

사용법: 자동으로 참조됨 (iOS 26 개발 시)

---

jungian-archetypes

한 줄 요약: 융의 12가지 원형(영웅, 마법사 등)을 UI 디자인 스타일로 변환하는 스킬입니다.

비유: "영웅적인 느낌의 디자인" 하면 대담하고 강렬한 색상, "현자 느낌" 하면 차분하고 깔끔한 색상 — 이런 성격-디자인 매칭표입니다.

언제 쓰나요? - 브랜드 성격에 맞는 디자인 방향을 잡을 때 - "이 앱은 어떤 느낌이어야 하지?"라는 질문에 답할 때

어떻게 작동하나요? 12가지 원형(영웅, 마법사, 탐험가, 반항아, 연인, 돌보는 자 등) 각각에 대해 시각 언어, 색상, 글꼴, 레이아웃, 애니메이션, 공간감 등을 정의합니다.

사용법: 디자인 방향 결정 시 참고

---

major-arcana

한 줄 요약: 타로 메이저 아르카나 22장을 색상 팔레트와 분위기로 변환한 스킬입니다.

비유: "이 화면은 '태양' 카드 같은 밝고 활기찬 느낌으로" 하면 해당 색상 팔레트를 자동으로 적용합니다.

언제 쓰나요? - 독특한 색상 팔레트를 찾을 때 - 감정/분위기 기반으로 색상을 정하고 싶을 때

어떻게 작동하나요? 22장의 카드 각각에 대해 주요 색상, 보조 색상, 강조 색상, 그라디언트, 그림자 스타일, 분위기를 정의합니다. "바보" = 하늘색+흰색(가벼움), "마법사" = 보라+은색(신비함) 식입니다.

사용법: 색상 팔레트 선택 시 참고

---

archetypal-combinations

한 줄 요약: 융 원형 + 타로 카드를 조합해서 독특한 디자인 시스템을 만드는 스킬입니다.

비유: "영웅 + 태양" = 대담하고 밝은 디자인, "마법사 + 달" = 신비롭고 어두운 디자인 — 이렇게 두 가지를 조합해서 새로운 디자인 레시피를 만듭니다.

언제 쓰나요? - 독특하면서도 일관된 디자인 시스템을 빠르게 만들 때 - "영웅+태양" 같은 공식으로 디자인 방향을 지정할 때

어떻게 작동하나요? 원형(구조/레이아웃/타이포그래피)과 카드(색상/분위기/조명)를 합성합니다. 원형에서 구조를, 카드에서 색상을 가져와서 완전한 Tailwind 설정 + 컴포넌트 예시를 만들어냅니다.

사용법: "Hero+Sun" 식으로 조합을 지정

---

frontend-slides

한 줄 요약: 외부 도구 없이 HTML만으로 멋진 프레젠테이션을 만드는 스킬입니다.

비유: 파워포인트 없이도 브라우저에서 바로 돌아가는 애니메이션 발표 자료를 만드는 것입니다.

언제 쓰나요? - 발표 자료를 만들 때 - PPT를 웹 프레젠테이션으로 변환할 때 - 디자인 감각이 부족한데 멋진 슬라이드가 필요할 때

어떻게 작동하나요? CSS와 JavaScript를 활용한 자체 프레젠테이션을 만듭니다. 외부 라이브러리 없이 HTML 하나로 완성됩니다. 다양한 스타일 프리셋이 있어서 취향에 맞는 디자인을 고를 수 있습니다.

사용법: "Create a presentation about..." 또는 "/slides"

---

7. 글쓰기/콘텐츠

글, 소셜 미디어, 영상 등 콘텐츠 제작을 돕는 스킬들입니다.

---

article-writing

한 줄 요약: 블로그, 가이드, 뉴스레터 같은 긴 글을 전문적으로 작성하는 스킬입니다.

비유: 전문 에디터가 글의 구조, 목소리, 증거를 다듬어주는 것과 같습니다.

언제 쓰나요? - 블로그 글이나 가이드를 쓸 때 - 특정 사람/브랜드의 문체를 따라 써야 할 때 - 노트나 메모를 정돈된 글로 바꿀 때

어떻게 작동하나요? 기존 글 예시를 주면 문장 길이, 문체, 수사법 등을 분석해서 그 목소리를 따라 씁니다. "오늘날 빠르게 변화하는..." 같은 AI 특유의 상투적 표현은 자동으로 삭제합니다. 모든 주장에 증거를 요구합니다.

사용법: "Write an article about..." 또는 "/write"

---

content-engine

한 줄 요약: 하나의 아이디어를 여러 플랫폼에 맞게 변환하는 콘텐츠 생산 엔진입니다.

비유: 같은 음식을 한식, 양식, 중식 스타일로 다르게 플레이팅하는 것처럼, 하나의 내용을 X, LinkedIn, YouTube 등 각 플랫폼에 맞게 변형합니다.

언제 쓰나요? - X 게시물이나 스레드를 쓸 때 - LinkedIn 글을 작성할 때 - 하나의 콘텐츠를 여러 플랫폼에 맞게 변환할 때

어떻게 작동하나요? 원본 아이디어/콘텐츠를 받아서, 각 플랫폼의 규칙(글자 수, 해시태그, 링크 처리)에 맞게 변환합니다. 각 플랫폼에 맞는 최적의 형태를 만들어줍니다.

사용법: "Write a post for X about..."

---

crosspost

한 줄 요약: 여러 소셜 플랫폼에 동시에 올리되, 각 플랫폼에 맞게 다르게 작성하는 스킬입니다.

비유: 같은 공지를 인스타, 페이스북, 트위터에 올리되, 각 플랫폼 사용자 취향에 맞게 조금씩 다르게 쓰는 것입니다.

언제 쓰나요? - 하나의 소식을 여러 플랫폼에 동시에 올릴 때 - 출시 공지, 업데이트 등을 배포할 때

어떻게 작동하나요? 핵심 규칙: 절대 동일한 내용을 복사-붙여넣기 하지 않습니다. X(280자), LinkedIn(3000자), Threads(500자), Bluesky(300자) 각각의 규격에 맞게 별도로 작성합니다. 기본 플랫폼에 먼저 올리고, 나머지를 적응시킵니다.

사용법: "crosspost this to..." 또는 `/crosspost`

---

x-api

한 줄 요약: X(Twitter)에 자동으로 글을 올리거나 데이터를 가져오는 스킬입니다.

비유: X의 "자동 비서"처럼, 프로그램이 대신 글을 올리고, 타임라인을 읽고, 검색합니다.

언제 쓰나요? - 프로그램에서 자동으로 X에 게시할 때 - X 데이터를 수집하거나 분석할 때 - X 봇이나 자동화를 만들 때

어떻게 작동하나요? X API를 사용해서 트윗 작성, 스레드 작성, 타임라인 읽기, 검색, 분석 등을 프로그램으로 제어합니다. OAuth 인증 방식과 속도 제한 관리 방법을 포함합니다.

사용법: 자동으로 참조됨 (X 관련 작업 시)

---

video-editing

한 줄 요약: 실제 촬영한 영상을 AI로 편집하는 워크플로우입니다.

비유: 촬영팀이 찍어온 원본 영상을 편집자가 잘라내고, 자막 넣고, 음악 붙이는 과정을 AI가 돕습니다.

언제 쓰나요? - 긴 녹화본을 짧은 영상으로 만들 때 - 자막, 배경음악, 오버레이를 추가할 때 - 유튜브, 틱톡, 인스타 등 플랫폼별 규격에 맞출 때

어떻게 작동하나요? Screen Studio로 촬영 > FFmpeg로 자르기 > Remotion으로 그래픽 추가 > ElevenLabs/fal.ai로 음성/효과 > Descript/CapCut으로 최종 편집. 각 도구가 하나의 역할을 담당합니다.

사용법: "edit this video..." 또는 "/video-editing"

---

videodb

한 줄 요약: 영상을 이해하고, 검색하고, 편집하는 종합 영상 처리 스킬입니다.

비유: 영상에 "00:15에서 고양이가 점프하는 장면"처럼 검색할 수 있게 만들고, 자막 추가, 음성 더빙 등도 할 수 있습니다.

언제 쓰나요? - 영상에서 특정 장면을 검색할 때 - 자막 생성/번역이 필요할 때 - 실시간 스트림을 모니터링할 때 - 데스크톱 화면을 녹화하고 분석할 때

어떻게 작동하나요? 영상을 분석해서 시각/음성/의미 인덱스를 만들고, 검색하면 정확한 타임스탬프와 클립을 제공합니다. 자막, 오버레이, 더빙, 배경음악 추가도 가능합니다. RTSP 라이브 스트림도 지원합니다.

사용법: 자동으로 참조됨 (영상 처리 작업 시)

---

fal-ai-media

한 줄 요약: AI로 이미지, 영상, 음성을 생성하는 스킬입니다.

비유: "이런 그림 그려줘", "이 이미지로 영상 만들어줘", "이 텍스트를 음성으로 바꿔줘"가 가능한 멀티미디어 제작 도구입니다.

언제 쓰나요? - AI로 이미지를 생성할 때 - 텍스트나 이미지를 영상으로 변환할 때 - 텍스트를 음성으로 변환할 때

어떻게 작동하나요? fal.ai MCP를 통해 다양한 AI 모델에 접근합니다. 텍스트>이미지(Nano Banana), 텍스트/이미지>영상(Seedance, Kling, Veo 3), 텍스트>음성(CSM-1B), 영상>오디오(ThinkSound) 등을 제공합니다.

사용법: “generate an image of...” 또는 “create a video...”

---

korean-cover-letter-writer

한 줄 요약: 한국어 자기소개서를 전문적으로 작성해주는 스킬입니다.

비유: 취업 컨설턴트가 자소서를 함께 작성해주는 것과 같습니다. 사용자의 실제 경험을 듣고, 인사담당자가 읽고 싶은 글로 구조화합니다.

언제 쓰나요? - 자기소개서, 커버레터를 쓸 때 - 지원동기서, 경력기술서를 작성할 때 - 기존 자소서를 첨삭받고 싶을 때

어떻게 작동하나요? 6단계 워크플로우: 1. 정보 수집 > 2. 채용공고 분석 > 3. 문항 의도 파악 > 4. 경험-직무 매칭 > 5. K-STAR-K 초안 작성 > 6. 다듬기. AI 특유의 문장 패턴을 철저히 배제하고, 정량적 근거(수치)를 강조합니다.

사용법: “자소서 써줘” 또는 “커버레터 도와줘”

---

visa-doc-translate

한 줄 요약: 비자 신청 서류를 영어로 번역하고 PDF로 만들어주는 스킬입니다.

비유: 공증 번역사가 원본 문서를 보고 영문 번역본을 만들어주는 것을 자동화한 것입니다.

언제 쓰나요? - 비자 신청용 서류를 영어로 번역할 때 - 예금증명서, 재직증명서 등을 영문으로 변환할 때

어떻게 작동하나요? 서류 이미지를 받으면 자동으로: 형식 변환 > 방향 교정 > OCR(글자 인식) > 전문 번역 > 원본+번역 PDF 생성. 인명은 한국어 원본 + 영문 병기합니다.

사용법: 이미지 파일 경로를 제공하면 자동 실행

---

8. 리서치

정보를 조사하고 분석하는 스킬들입니다.

---

deep-research

한 줄 요약: 여러 웹 소스를 검색해서 출처가 있는 연구 보고서를 만드는 스킬입니다.

비유: 대학원생이 논문을 쓰기 위해 여러 자료를 찾아 읽고, 출처를 밝히면서 보고서를 작성하는 것과 같습니다.

언제 쓰나요? - 특정 주제를 깊이 조사할 때 - 기술 평가, 경쟁 분석, 시장 조사가 필요할 때 - 근거 있는 보고서가 필요할 때

어떻게 작동하나요? 주제를 3~5개 세부 질문으로 나눈 후, 각 질문에 대해 firecrawl과 exa MCP로 웹 검색합니다. 여러 소스의 정보를 종합하고, 출처를 명시한 보고서를 작성합니다. 모든 주장에 근거를 요구합니다.

사용법: “research...” 또는 “deep dive into...”

---

exa-search

한 줄 요약: Exa AI의 신경망 검색으로 웹, 코드, 회사, 사람을 검색하는 스킬입니다.

비유: 구글보다 더 의미를 이해하는 AI 검색 엔진입니다. "이 개념과 비슷한 것"을 찾는 데 강합니다.

언제 쓰나요? - 최신 웹 정보가 필요할 때 - 코드 예시나 기술 문서를 찾을 때 - 회사나 사람을 조사할 때

어떻게 작동하나요? Exa MCP를 통해 신경망 기반 검색을 수행합니다. 일반 키워드 검색보다 의미를 더 잘 이해해서 관련성 높은 결과를 가져옵니다.

사용법: "search for..." 또는 "look up..."

---

market-research

한 줄 요약: 시장 조사, 경쟁 분석, 투자자 조사를 전문적으로 수행하는 스킬입니다.

비유: 전문 리서치 회사에 의뢰하는 것처럼, "이 시장은 얼마나 크고, 경쟁사는 누가 있고, 트렌드는 뭔지"를 조사합니다.

언제 쓰나요? - 새 시장에 진출하기 전 조사할 때 - 경쟁사를 비교 분석할 때 - 투자자/펀드를 조사할 때 - 시장 규모(TAM/SAM/SOM)를 추정할 때

어떻게 작동하나요? 모든 주장에 출처를 요구하고, 오래된 데이터는 표시합니다. 반대 증거도 포함하고, 단순 요약이 아닌 "의사결정에 도움이 되는" 분석을 제공합니다.

사용법: "research the market for..." 또는 "/market-research"

---

product-lens

한 줄 요약: "이걸 만들어야 하나?" 판단을 도와주는 제품 전략 검토 스킬입니다.

비유: Y Combinator 오피스 아워처럼 "이 제품의 타겟은 누구인지, 왜 지금인지, MVP는 뭔지" 날카로운 질문을 던집니다.

언제 쓰나요? - 새 기능을 시작하기 전에 "왜?"를 검증할 때 - 어떤 기능을 먼저 만들지 결정할 때 - 주간 제품 리뷰를 할 때

어떻게 작동하나요? "진단 모드": 7가지 핵심 질문(누구를 위한 것인지, 고통은 뭔지, 왜 지금인지 등)으로 제품을 점검합니다. "창업자 리뷰 모드": 프로젝트 코드를 읽고 제품-시장 적합성을 0~10으로 평가합니다.

사용법: `/product-lens` 또는 자동 활성화

---

data-scraper-agent

한 줄 요약: 웹사이트의 공개 데이터를 자동으로 수집하는 봇을 만드는 스킬입니다.

비유: 매일 특정 사이트를 방문해서 가격/채용정보/뉴스를 메모해주는 비서 로봇을 만드는 것입니다.

언제 쓰나요? - 특정 사이트를 정기적으로 모니터링하고 싶을 때 - 가격, 채용공고, 뉴스 등을 자동 수집할 때 - 무료 도구만으로 데이터 수집 봇을 만들고 싶을 때

어떻게 작동하나요? 수집(Scraper) > 분석(AI가 분류/요약) > 저장(Notion/Google Sheets/Supabase) 3단계로 작동합니다. GitHub Actions로 무료 자동 실행되고, Gemini Flash로 무료 AI 분석이 가능합니다. 사용자 피드백으로 점점 더 정확해집니다.

사용법: "build a bot that checks..."

---

9. 프로젝트 관리

프로젝트 계획, 투자 자료, 비즈니스 문서를 만드는 스킬들입니다.

---

investor-materials

한 줄 요약: 투자자용 자료(피치덱, 재무 모델 등)를 일관되게 만드는 스킬입니다.

비유: 투자 유치를 위한 발표 자료, 재무 계획서, 사업 요약서를 "같은 숫자, 같은 이야기"로 통일해서 만드는 것입니다.

언제 쓰나요? - 피치덱을 만들거나 수정할 때 - 투자자 메모, 한 장 요약을 작성할 때 - 재무 모델이나 자금 사용 계획을 만들 때 - 액셀러레이터 지원서를 쓸 때

어떻게 작동하나요? 핵심 원칙: 모든 투자 자료의 숫자와 내용이 서로 일치해야 합니다. 먼저 단일 진실 소스(핵심 지표, 가격, 팀 정보)를 확정한 후, 각 자료를 이에 맞춰 작성합니다.

사용법: "create a pitch deck..." 또는 "/investor-materials"

---

investor-outreach

한 줄 요약: 투자자에게 보내는 이메일과 메시지를 작성하는 스킬입니다.

비유: 투자자 한 명 한 명에게 맞춤형 편지를 쓰는 것과 같습니다. 복사-붙여넣기 금지, 모든 메일을 개인화합니다.

언제 쓰나요? - 투자자에게 콜드 이메일을 보낼 때 - 미팅 후 팔로업 메일을 보낼 때 - 투자 진행 중 업데이트를 보낼 때

어떻게 작동하나요? 투자자의 포트폴리오, 투자 이론, 최근 활동 등을 참고해서 개인화된 메시지를 작성합니다. 간결하고, 증거 기반이며, 한 가지 구체적 다음 행동을 요청합니다.

사용법: "write an investor email to..."

---

10. 언어별 도구

특정 프로그래밍 언어나 프레임워크에 특화된 스킬들입니다. 각 언어의 "이렇게 쓰면 좋다"는 모범 사례가 담겨 있습니다.

TypeScript / JavaScript

bun-runtime

한 줄 요약: Bun이라는 빠른 JavaScript 실행 도구의 사용법입니다.

비유: Node.js의 "터보 버전"이라고 보면 됩니다. 설치, 실행, 테스트가 훨씬 빠릅니다.

언제 쓰나요? - 새 JS/TS 프로젝트에서 빠른 도구를 쓰고 싶을 때 - Node.js에서 Bun으로 전환할 때

어떻게 작동하나요? 기존 Node.js 명령어를 거의 그대로 쓸 수 있으면서, 설치와 실행이 훨씬 빠릅니다. bun install(패키지 설치), bun run(실행), bun test(테스트) 등을 제공합니다.

사용법: 자동으로 참조됨 (Bun 관련 작업 시)

---

nextjs-turbopack

한 줄 요약: Next.js 16+에서 기본 사용되는 빠른 빌드 도구(Turbopack)의 사용법입니다.

비유: Next.js의 개발 속도를 5~14배 빠르게 해주는 "가속 장치"입니다.

언제 쓰나요? - Next.js 16+ 앱을 개발할 때 - 개발 서버 시작이 느릴 때

어떻게 작동하나요? Next.js 16부터 기본 탑재된 번들러로, Rust로 만들어져서 빠릅니다. 파일 시스템 캐시를 써서 재시작해도 이전 작업을 재활용합니다.

사용법: 자동으로 참조됨 (Next.js 개발 시)

---

nuxt4-patterns

한 줄 요약: Nuxt 4(Vue 기반 프레임워크)의 SSR, 데이터 가져오기, 라우팅 패턴입니다.

비유: Vue.js로 서버와 클라이언트 양쪽에서 작동하는 앱을 만드는 노하우 모음입니다.

언제 쓰나요? - Nuxt 4 앱을 만들거나 디버그할 때 - 서버/클라이언트 간 데이터 불일치 문제를 해결할 때

어떻게 작동하나요? 서버에서 미리 만든 페이지와 클라이언트에서 동작하는 페이지가 일치하도록 하는 방법, 데이터를 안전하게 가져오는 방법, 경로별 렌더링 전략을 제공합니다.

사용법: 자동으로 참조됨 (Nuxt 4 개발 시)

Python

python-patterns

한 줄 요약: Python 코드를 깔끔하고 효율적으로 작성하는 모범 사례입니다.

비유: "파이썬스러운" 코드를 작성하는 방법입니다. 다른 언어 습관 대신 Python만의 장점을 활용합니다.

언제 쓰나요? - Python 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? 타입 힌트, 패턴 매칭, 비동기 처리, 데이터 클래스, 에러 처리 등 현대 Python의 핵심 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Python 작업 시)

---

python-testing

한 줄 요약: Python 테스트를 작성하는 방법(pytest 기반)입니다.

비유: Python 코드가 제대로 작동하는지 확인하는 시험 문제를 만드는 방법입니다.

언제 쓰나요? - Python 테스트를 작성하거나 디버그할 때

어떻게 작동하나요? pytest 프레임워크를 사용한 단위 테스트, 통합 테스트, fixture 설정, mock 사용법, 커버리지 측정 방법을 제공합니다.

사용법: 자동으로 참조됨 (Python 테스트 시)

---

pytorch-patterns

한 줄 요약: AI 모델을 만드는 PyTorch 프레임워크의 사용 패턴입니다.

비유: AI 요리사의 레시피북처럼, 딥러닝 모델을 효율적으로 학습시키는 방법이 담겨 있습니다.

언제 쓰나요? - PyTorch로 AI 모델을 만들거나 학습시킬 때 - GPU 메모리 최적화가 필요할 때

어떻게 작동하나요? 디바이스 독립적 코드, 재현성 보장, 학습 루프, 데이터 로딩, 모델 저장 등의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (PyTorch 작업 시)

Go

golang-patterns

한 줄 요약: Go 언어의 코딩 패턴과 모범 사례입니다.

비유: Go 언어만의 규칙과 관습을 정리한 "Go 매너 교본"입니다.

언제 쓰나요? - Go 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? 에러 처리, 인터페이스, 동시성(goroutine/channel), 프로젝트 구조, 패키지 설계 등 Go만의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Go 작업 시)

---

golang-testing

한 줄 요약: Go 언어의 테스트 작성 방법입니다.

비유: Go 코드의 시험 문제를 만드는 방법입니다.

언제 쓰나요? - Go 테스트를 작성하거나 디버그할 때

어떻게 작동하나요? Go의 내장 테스트 프레임워크, 테이블 기반 테스트, mock, 벤치마크, 통합 테스트 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Go 테스트 시)

Rust

rust-patterns

한 줄 요약: Rust 언어의 소유권, 에러 처리, 동시성 등 핵심 패턴입니다.

비유: Rust 특유의 "메모리 안전성"을 지키면서도 효율적인 코드를 쓰는 방법입니다.

언제 쓰나요? - Rust 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? 소유권과 빌림, Result/Option 기반 에러 처리, 트레이트, 비동기 프로그래밍 등 Rust의 핵심 개념을 패턴으로 정리합니다.

사용법: 자동으로 참조됨 (Rust 작업 시)

---

rust-testing

한 줄 요약: Rust 테스트 작성 방법입니다.

비유: Rust 코드가 안전하게 작동하는지 확인하는 방법입니다.

언제 쓰나요? - Rust 테스트를 작성하거나 디버그할 때

어떻게 작동하나요? 단위 테스트, 통합 테스트, 프로퍼티 기반 테스트, 벤치마크 등의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Rust 테스트 시)

Java / Spring Boot

java-coding-standards

한 줄 요약: Java 코딩 표준(이름 짓기, 불변성, 예외 처리 등)입니다.

비유: Java 코드를 깔끔하게 쓰는 규칙집입니다.

언제 쓰나요? - Java/Spring Boot 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? 명확한 이름 짓기, 불변 객체 선호, 적절한 예외 처리, Optional 사용법, 스트림 활용, 패키지 구조 등을 정리합니다.

사용법: 자동으로 참조됨 (Java 작업 시)

---

jpa-patterns

한 줄 요약: Java 데이터베이스 접근(JPA/Hibernate)의 패턴입니다.

비유: Java에서 데이터베이스를 효율적으로 사용하는 방법(N+1 문제 해결 등)입니다.

언제 쓰나요? - JPA 엔티티 설계, 쿼리 최적화, 트랜잭션 관리 시

어떻게 작동하나요? 엔티티 설계, 관계 매핑, N+1 문제 방지, 페이지네이션, 캐싱, 커넥션 풀링 등을 제공합니다.

사용법: 자동으로 참조됨 (JPA 작업 시)

---

springboot-patterns

한 줄 요약: Spring Boot의 아키텍처와 API 설계 패턴입니다.

비유: Spring Boot로 서버를 만들 때의 설계 도면입니다.

언제 쓰나요? - Spring Boot REST API를 만들 때

어떻게 작동하나요? 컨트롤러 > 서비스 > 리포지토리 계층 구조, 검증, 예외 처리, 캐싱, 비동기 처리, 이벤트 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Spring Boot 작업 시)

---

springboot-security

한 줄 요약: Spring Boot의 보안 설정(인증, 인가, CORS 등) 모범 사례입니다.

비유: Spring Boot 앱의 보안 잠금장치를 설정하는 매뉴얼입니다.

언제 쓰나요? - Spring Security 설정 시

어떻게 작동하나요? JWT 인증, OAuth2, 역할 기반 접근 제어, 입력 검증, CSRF 보호, 시크릿 관리, 속도 제한 등을 제공합니다.

사용법: 자동으로 참조됨 (Spring 보안 작업 시)

---

springboot-tdd

한 줄 요약: Spring Boot 프로젝트의 테스트 주도 개발 방법입니다.

비유: Spring Boot 코드의 시험 문제를 먼저 만들고 코드를 작성하는 방법입니다.

언제 쓰나요? - Spring Boot에서 새 기능을 만들거나 버그를 고칠 때

어떻게 작동하나요? JUnit 5 + Mockito로 단위 테스트, MockMvc로 API 테스트, Testcontainers로 통합 테스트, JaCoCo로 커버리지를 측정합니다.

사용법: 자동으로 참조됨 (Spring Boot TDD 시)

---

springboot-verification

한 줄 요약: Spring Boot 프로젝트의 배포 전 전체 검증 과정입니다.

비유: Spring Boot 앱의 출시 전 품질 검사 체크리스트입니다.

언제 쓰나요? - PR을 만들기 전 - 프로덕션 배포 전

어떻게 작동하나요? 빌드 > 정적 분석 > 테스트+커버리지 > 보안 스캔 > 변경사항 리뷰 순으로 진행합니다.

사용법: 자동으로 참조됨 (Spring Boot 배포 전)

Kotlin

kotlin-patterns

한 줄 요약: Kotlin 언어의 관용적 패턴(null 안전, 코루틴, DSL 등)입니다.

비유: Kotlin만의 우아한 코드 작성법을 담은 스타일 가이드입니다.

언제 쓰나요? - Kotlin 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? null 안전, 불변성, sealed class, 코루틴, 확장 함수, DSL 빌더, Gradle 설정 등의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Kotlin 작업 시)

---

kotlin-coroutines-flows

한 줄 요약: Kotlin의 비동기 처리(코루틴)와 데이터 스트림(Flow) 패턴입니다.

비유: 여러 작업을 동시에 효율적으로 처리하는 Kotlin만의 방법입니다.

언제 쓰나요? - 비동기 코드를 작성할 때 - Flow, StateFlow, SharedFlow를 사용할 때

어떻게 작동하나요? 구조적 동시성, Flow 연산자, 에러 처리, 취소 관리, 테스트 방법을 제공합니다.

사용법: 자동으로 참조됨 (Kotlin 비동기 작업 시)

---

kotlin-exposed-patterns

한 줄 요약: JetBrains Exposed ORM으로 데이터베이스를 다루는 패턴입니다.

비유: Kotlin에서 데이터베이스를 효율적으로 사용하는 방법입니다.

언제 쓰나요? - Exposed로 데이터베이스 작업을 할 때

어떻게 작동하나요? DSL 쿼리, DAO 패턴, HikariCP 커넥션 풀링, Flyway 마이그레이션, 리포지토리 패턴 등을 제공합니다.

사용법: 자동으로 참조됨 (Exposed 작업 시)

---

kotlin-ktor-patterns

한 줄 요약: Ktor(Kotlin HTTP 서버) 프레임워크의 사용 패턴입니다.

비유: Kotlin으로 웹 서버를 만들 때의 설계 도면입니다.

언제 쓰나요? - Ktor 서버를 만들거나 설정할 때

어떻게 작동하나요? 라우팅 DSL, 플러그인 설정, 인증, Koin DI, WebSocket, testApplication 테스트 등을 제공합니다.

사용법: 자동으로 참조됨 (Ktor 작업 시)

---

kotlin-testing

한 줄 요약: Kotlin 테스트 작성 방법(Kotest + MockK)입니다.

비유: Kotlin 코드의 시험 문제를 만드는 방법입니다.

언제 쓰나요? - Kotlin 테스트를 작성할 때

어떻게 작동하나요? Kotest 스펙 스타일, MockK 모킹, 프로퍼티 기반 테스트, Kover 커버리지 등을 제공합니다.

사용법: 자동으로 참조됨 (Kotlin 테스트 시)

Swift / Apple

swiftui-patterns

한 줄 요약: SwiftUI(Apple 앱 UI 프레임워크)의 아키텍처 패턴입니다.

비유: Apple 앱의 화면을 깔끔하게 만드는 방법입니다.

언제 쓰나요? - SwiftUI 뷰와 상태를 관리할 때 - 네비게이션, 성능 최적화를 할 때

어떻게 작동하나요? @State, @Observable, @Binding 선택법, NavigationStack, 환경 값, 리스트 최적화 등을 제공합니다.

사용법: 자동으로 참조됨 (SwiftUI 작업 시)

---

swift-concurrency-6-2

한 줄 요약: Swift 6.2의 새로운 동시성 모델(기본 단일 스레드)입니다.

비유: Swift 6.2에서 "기본적으로 안전하게, 필요할 때만 병렬로" 코드를 작성하는 방법입니다.

언제 쓰나요? - Swift 6.2로 마이그레이션할 때 - 데이터 경쟁 에러를 해결할 때

어떻게 작동하나요? 기본적으로 단일 스레드에서 실행되어 안전하고, 무거운 작업만 @concurrent로 표시해서 백그라운드에서 실행합니다.

사용법: 자동으로 참조됨 (Swift 6.2 작업 시)

---

swift-actor-persistence

한 줄 요약: Swift의 Actor를 사용한 스레드 안전 데이터 저장 패턴입니다.

비유: 여러 명이 동시에 접근해도 안전한 금고 시스템을 만드는 방법입니다.

언제 쓰나요? - 스레드 안전한 데이터 저장이 필요할 때 - 오프라인 우선 앱을 만들 때

어떻게 작동하나요? Actor가 데이터 접근을 직렬화해서 동시 접근 시 충돌을 방지합니다. 메모리 캐시 + 파일 저장을 조합합니다.

사용법: 자동으로 참조됨 (Swift 저장 작업 시)

---

swift-protocol-di-testing

한 줄 요약: Swift에서 프로토콜 기반 의존성 주입으로 테스트 가능한 코드를 만드는 패턴입니다.

비유: 테스트할 때 실제 인터넷/파일 접근 대신 "가짜" 대역을 넣는 방법입니다.

언제 쓰나요? - 파일, 네트워크 등 외부 시스템에 의존하는 코드를 테스트할 때

어떻게 작동하나요? 외부 의존성을 프로토콜 뒤에 숨기고, 테스트에서는 프로토콜을 구현한 가짜 객체를 사용합니다.

사용법: 자동으로 참조됨 (Swift 테스트 시)

---

foundation-models-on-device

한 줄 요약: Apple 기기에서 직접 AI 모델을 실행하는 방법(iOS 26+)입니다.

비유: 클라우드 없이 아이폰 안에서 직접 AI가 작동하는 기능입니다. 인터넷이 없어도 되고, 데이터가 기기 밖으로 나가지 않습니다.

언제 쓰나요? - Apple Intelligence를 앱에 통합할 때 - 클라우드 의존 없는 AI 기능이 필요할 때

어떻게 작동하나요? FoundationModels 프레임워크를 통해 텍스트 생성, 구조화된 출력(@Generable), 도구 호출, 스트리밍을 기기에서 직접 실행합니다.

사용법: 자동으로 참조됨 (Apple AI 작업 시)

Android / Kotlin Multiplatform

android-clean-architecture

한 줄 요약: Android 앱의 클린 아키텍처(모듈 분리, 의존성 규칙)입니다.

비유: Android 앱을 깔끔하게 정리하는 방법 — 마치 서랍장에 종류별로 물건을 분류하듯이.

언제 쓰나요? - Android/KMP 프로젝트 구조를 잡을 때 - UseCase, Repository 패턴을 구현할 때

어떻게 작동하나요? app(진입점) > domain(비즈니스 로직) > data(데이터 접근) > presentation(UI) 으로 모듈을 나누고, 의존성은 안쪽으로만 흐르게 합니다.

사용법: 자동으로 참조됨 (Android 개발 시)

---

compose-multiplatform-patterns

한 줄 요약: Compose Multiplatform(여러 플랫폼 공유 UI)의 패턴입니다.

비유: 하나의 코드로 Android, iOS, 데스크톱, 웹 화면을 동시에 만드는 방법입니다.

언제 쓰나요? - Compose UI를 여러 플랫폼에서 공유할 때 - 상태 관리, 네비게이션, 테마를 설정할 때

어떻게 작동하나요? 단일 상태 객체, ViewModel, StateFlow 패턴으로 상태를 관리하고, 플랫폼별 UI 차이를 처리하는 방법을 제공합니다.

사용법: 자동으로 참조됨 (Compose Multiplatform 작업 시)

Flutter

flutter-dart-code-review

한 줄 요약: Flutter/Dart 코드 리뷰 체크리스트입니다.

비유: Flutter 앱의 코드를 검수하는 품질 체크리스트입니다.

언제 쓰나요? - Flutter 코드를 리뷰할 때

어떻게 작동하나요? 프로젝트 건강, Dart 언어 함정, 위젯 모범 사례, 상태 관리(BLoC, Riverpod 등), 성능, 접근성, 보안, 클린 아키텍처를 체크리스트로 검사합니다.

사용법: 자동으로 참조됨 (Flutter 리뷰 시)

C++

cpp-coding-standards

한 줄 요약: 현대 C++(C++17/20/23)의 코딩 표준입니다.

비유: C++ 코드를 안전하고 깔끔하게 쓰는 규칙집(C++ Core Guidelines 기반)입니다.

언제 쓰나요? - C++ 코드를 작성하거나 리뷰할 때

어떻게 작동하나요? 타입 안전, 리소스 안전(스마트 포인터), 불변성, enum class, 이동 의미론 등의 규칙을 제공합니다.

사용법: 자동으로 참조됨 (C++ 작업 시)

---

cpp-testing

한 줄 요약: C++ 테스트 작성 방법(GoogleTest + CMake)입니다.

비유: C++ 코드의 안전성을 확인하는 테스트를 만드는 방법입니다.

언제 쓰나요? - C++ 테스트를 작성하거나 디버그할 때

어떻게 작동하나요? GoogleTest/GoogleMock, CTest 워크플로우, sanitizer 설정, flaky 테스트 대응 방법을 제공합니다.

사용법: 자동으로 참조됨 (C++ 테스트 시)

Perl

perl-patterns

한 줄 요약: 현대 Perl 5.36+의 코딩 패턴입니다.

비유: Perl 코드를 현대적이고 깔끔하게 쓰는 방법입니다.

언제 쓰나요? - Perl 코드를 작성하거나 리뷰할 때 - 레거시 Perl을 현대화할 때

어떻게 작동하나요? use v5.36 프래그마, 서브루틴 시그니처, 에러 처리, 모듈 구조 등의 현대 Perl 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Perl 작업 시)

---

perl-security

한 줄 요약: Perl 애플리케이션의 보안 모범 사례입니다.

비유: Perl 앱의 보안 잠금장치를 설정하는 방법입니다.

언제 쓰나요? - Perl에서 사용자 입력을 처리하거나 보안 설정을 할 때

어떻게 작동하나요? 입력 검증(taint mode), SQL 인젝션 방지, XSS 방지, 파일 처리 보안 등을 제공합니다.

사용법: 자동으로 참조됨 (Perl 보안 작업 시)

---

perl-testing

한 줄 요약: Perl 테스트 작성 방법입니다.

비유: Perl 코드의 시험 문제를 만드는 방법입니다.

언제 쓰나요? - Perl 테스트를 작성할 때

어떻게 작동하나요? Test::More, Test::Exception, mock, 테스트 구조 등을 제공합니다.

사용법: 자동으로 참조됨 (Perl 테스트 시)

Django (Python 웹 프레임워크)

django-patterns

한 줄 요약: Django 웹 프레임워크의 아키텍처 패턴입니다.

비유: Python Django로 웹 앱을 만들 때의 설계 도면입니다.

언제 쓰나요? - Django 앱을 만들거나 구조를 잡을 때

어떻게 작동하나요? 모델 설계, 뷰 구조, 템플릿 패턴, 폼 처리, 시리얼라이저, 미들웨어 등을 제공합니다.

사용법: 자동으로 참조됨 (Django 작업 시)

---

django-security

한 줄 요약: Django 앱의 보안 설정 모범 사례입니다.

비유: Django 앱의 보안 잠금장치 설정 매뉴얼입니다.

언제 쓰나요? - Django 인증/인가, 보안 헤더, CSRF 보호를 설정할 때

어떻게 작동하나요? 프로덕션 보안 설정, 인증 시스템, 비밀번호 정책, CSRF/XSS 방지, 보안 헤더 설정 등을 제공합니다.

사용법: 자동으로 참조됨 (Django 보안 작업 시)

---

django-tdd

한 줄 요약: Django 프로젝트의 테스트 주도 개발 방법입니다.

비유: Django 코드의 시험 문제를 먼저 만들고 코드를 작성하는 방법입니다.

언제 쓰나요? - Django에서 새 기능을 만들거나 버그를 고칠 때

어떻게 작동하나요? Factory Boy, pytest-django를 사용한 모델/뷰/API 테스트, 커버리지 80%+ 목표의 TDD 워크플로우를 제공합니다.

사용법: 자동으로 참조됨 (Django TDD 시)

---

django-verification

한 줄 요약: Django 프로젝트의 배포 전 전체 검증 과정입니다.

비유: Django 앱의 출시 전 품질 검사 체크리스트입니다.

언제 쓰나요? - PR을 만들기 전 - 프로덕션 배포 전

어떻게 작동하나요? 환경 체크 > 린트/정적 분석 > 테스트+커버리지 > 보안 > 마이그레이션 > 배포 준비 순으로 검증합니다.

사용법: 자동으로 참조됨 (Django 배포 전)

Laravel (PHP 웹 프레임워크)

laravel-patterns

한 줄 요약: Laravel 웹 프레임워크의 아키텍처 패턴입니다.

비유: PHP Laravel로 웹 앱을 만들 때의 설계 도면입니다.

언제 쓰나요? - Laravel 앱을 만들거나 구조를 잡을 때

어떻게 작동하나요? 컨트롤러 > 서비스/액션 > 모델 구조, Eloquent ORM, 큐, 이벤트, 캐싱 패턴을 제공합니다.

사용법: 자동으로 참조됨 (Laravel 작업 시)

---

laravel-security

한 줄 요약: Laravel 앱의 보안 설정 모범 사례입니다.

비유: Laravel 앱의 보안 잠금장치 설정 매뉴얼입니다.

언제 쓰나요? - Laravel 인증/인가, 검증, CSRF 보호를 설정할 때

어떻게 작동하나요? 미들웨어, Guard/Policy, Form Request, 속도 제한, 암호화, 서명된 URL 등의 보안 메커니즘을 제공합니다.

사용법: 자동으로 참조됨 (Laravel 보안 작업 시)

---

laravel-tdd

한 줄 요약: Laravel 프로젝트의 테스트 주도 개발 방법입니다.

비유: Laravel 코드의 시험 문제를 먼저 만들고 코드를 작성하는 방법입니다.

언제 쓰나요? - Laravel에서 새 기능을 만들거나 버그를 고칠 때

어떻게 작동하나요? PHPUnit과 Pest를 사용한 단위/기능 테스트, Factory, 데이터베이스 테스트, 커버리지 80%+ 목표의 TDD 워크플로우를 제공합니다.

사용법: 자동으로 참조됨 (Laravel TDD 시)

---

laravel-verification

한 줄 요약: Laravel 프로젝트의 배포 전 전체 검증 과정입니다.

비유: Laravel 앱의 출시 전 품질 검사 체크리스트입니다.

언제 쓰나요? - PR을 만들기 전 - 프로덕션 배포 전

어떻게 작동하나요? 환경 > Composer > 린트/정적분석 > 테스트+커버리지 > 보안 > 마이그레이션 > 배포 준비 순으로 검증합니다.

사용법: 자동으로 참조됨 (Laravel 배포 전)

ClickHouse

clickhouse-io

한 줄 요약: ClickHouse(고성능 분석 데이터베이스)의 사용 패턴입니다.

비유: 수십억 건의 데이터를 빠르게 분석하는 전용 데이터베이스의 사용법입니다.

언제 쓰나요? - ClickHouse 테이블 설계, 쿼리 최적화, 대량 데이터 처리 시

어떻게 작동하나요? MergeTree 엔진, 파티셔닝, 프로젝션, 물리화 뷰, Kafka 연동 등의 패턴을 제공합니다.

사용법: 자동으로 참조됨 (ClickHouse 작업 시)

---

11. 특수 목적

특정 산업이나 업무에 특화된 전문 지식 스킬들입니다.

---

carrier-relationship-management

한 줄 요약: 운송 업체(캐리어)를 관리하고 계약을 협상하는 전문 지식입니다.

비유: 15년 경력 운송 관리자의 노하우가 담겨 있습니다. 새 업체 검증, 입찰 진행, 성과 평가, 요율 협상 방법 등입니다.

언제 쓰나요? - 새 운송 업체를 평가하고 계약할 때 - 연간 입찰(RFP)을 진행할 때 - 운송 성과를 스코어카드로 관리할 때

어떻게 작동하나요? 업체 검증(FMCSA 안전 기록, 보험 확인) > RFP 구조화 > 요율 분해 협상 > 라우팅 가이드 작성 > 성과 추적의 전체 라이프사이클을 안내합니다.

사용법: 운송 관리 업무 시 참고

---

customs-trade-compliance

한 줄 요약: 수출입 관세, 통관, 무역 규정 준수 전문 지식입니다.

비유: 15년 경력 무역 규정 전문가의 노하우입니다. 관세 분류, 서류 작성, 제재 대상 심사 방법 등입니다.

언제 쓰나요? - 상품의 HS/HTS 관세 코드를 분류할 때 - 통관 서류를 준비할 때 - 거래 상대방이 제재 대상인지 심사할 때 - FTA 혜택을 활용할 때

어떻게 작동하나요? GRI 규칙에 따른 상품 분류 > 관세율 결정 > 거래 당사자 심사 > 서류 준비 및 검증 순으로 진행합니다.

사용법: 무역 규정 업무 시 참고

---

energy-procurement

한 줄 요약: 전기/가스 구매, 요금 최적화, 재생에너지 계약 전문 지식입니다.

비유: 대기업의 에너지 구매 팀장이 가진 노하우입니다. 연간 수십억 원의 에너지 비용을 관리하는 방법입니다.

언제 쓰나요? - 전기/가스 공급 업체 입찰을 진행할 때 - 요금 구조를 분석하고 최적화할 때 - 태양광 PPA(전력 구매 계약)를 평가할 때 - 에너지 예산을 편성할 때

어떻게 작동하나요? 시설별 전력 사용 패턴 분석 > 요금 구조 최적화 > 공급 RFP > 재생에너지 도입 > 예산/헤지 전략 순으로 안내합니다.

사용법: 에너지 관리 업무 시 참고

---

production-scheduling

한 줄 요약: 제조 공장의 생산 스케줄링 전문 지식입니다.

비유: 공장의 생산 라인을 어떤 순서로, 어떤 제품을 만들지 결정하는 15년 경력 스케줄러의 노하우입니다.

언제 쓰나요? - 작업 순서를 최적화할 때 - 설비 고장이나 자재 부족 시 스케줄을 재편할 때 - 병목 공정을 파악하고 해소할 때

어떻게 작동하나요? 병목(제약) 식별 > 우선순위 분류 > 디스패칭 룰 적용 > 셋업 최적화 > 시프트별 실행 계획 순으로 진행합니다.

사용법: 생산 계획 업무 시 참고

---

inventory-demand-planning

한 줄 요약: 수요 예측, 안전 재고, 발주 계획 전문 지식입니다.

비유: 마트에서 "이 상품 얼마나 주문해야 하지?"를 결정하는 15년 경력 수요 기획자의 노하우입니다.

언제 쓰나요? - 수요 예측 모델을 선택하거나 검토할 때 - 안전 재고 수준을 결정할 때 - 프로모션이나 시즌 전환 계획을 세울 때 - 예측 정확도를 개선할 때

어떻게 작동하나요? 판매 데이터 수집 > ABC/XYZ 분류별 예측 모델 선택 > 프로모션 효과 반영 > 안전 재고 계산 > 발주 의사결정 순으로 진행합니다.

사용법: 재고/수요 관리 업무 시 참고

---

logistics-exception-management

한 줄 요약: 배송 지연, 분실, 파손 등 물류 예외 상황 처리 전문 지식입니다.

비유: "물건이 안 왔어요!", "파손되었어요!" 같은 상황을 체계적으로 해결하는 15년 경력 물류 전문가의 노하우입니다.

언제 쓰나요? - 배송 지연, 분실, 파손이 발생했을 때 - 운송사와 분쟁이 있을 때 - 화물 클레임을 접수하거나 관리할 때

어떻게 작동하나요? 예외 분류(지연/파손/분실/거부) > 심각도 판단 > 해결 워크플로우 적용 > 증거 수집 > 에스컬레이션 > 클레임 접수 순으로 진행합니다.

사용법: 물류 예외 처리 업무 시 참고

---

quality-nonconformance

한 줄 요약: 제조업 품질 관리와 부적합 조사 전문 지식입니다.

비유: "이 부품이 규격에 안 맞는데 어떻게 하지?"를 체계적으로 해결하는 15년 경력 품질 엔지니어의 노하우입니다.

언제 쓰나요? - 부적합(NCR)을 조사할 때 - 근본 원인 분석(5-Why, 이시카와)을 할 때 - 시정/예방 조치(CAPA)를 수립할 때 - SPC(통계적 공정 관리) 데이터를 해석할 때

어떻게 작동하나요? 부적합 감지 > 격리 > 심각도 분류 > 근본 원인 분석 > 처리(사용, 수리, 폐기) > CAPA 수립 순으로 진행합니다.

사용법: 품질 관리 업무 시 참고

---

returns-reverse-logistics

한 줄 요약: 반품 처리, 검수, 환불, 사기 탐지 전문 지식입니다.

비유: "반품하고 싶어요"에서 시작해서 검수, 등급 매기기, 재입고/폐기 결정까지의 전체 과정을 관리하는 노하우입니다.

언제 쓰나요? - 반품 요청을 처리할 때 - 반품 검수와 등급 분류를 할 때 - 반품 사기를 탐지할 때 - 보증 클레임을 관리할 때

어떻게 작동하나요? 반품 자격 확인 > RMA 발급 > 검수 및 등급 부여(A~D) > 처리 방법 결정(재입고/리퍼/폐기) > 환불 > 사기 패턴 감시 순으로 진행합니다.

사용법: 반품 관리 업무 시 참고

---

nutrient-document-processing

한 줄 요약: 문서를 변환, OCR, 워터마크, 서명, 편집하는 API 활용법입니다.

비유: PDF, Word, Excel 등 문서를 자유자재로 변환하고 처리하는 만능 문서 처리 도구입니다.

언제 쓰나요? - PDF를 Word로, Word를 PDF로 변환할 때 - 스캔 문서에서 텍스트를 추출(OCR)할 때 - 문서에 워터마크, 서명을 추가할 때

어떻게 작동하나요? Nutrient API에 파일을 보내고 원하는 처리 방식을 지정하면 결과를 받습니다. 변환, OCR, 테이블 추출, 개인정보 삭제, 서명 등 다양한 기능을 제공합니다.

사용법: 문서 처리 파이프라인에서 사용

---

clickhouse-io

(위 “언어별 도구 > ClickHouse” 섹션에서 설명됨)

---

compose-multiplatform-patterns

(위 “언어별 도구 > Android / Kotlin Multiplatform” 섹션에서 설명됨)

---

foundation-models-on-device

(위 “언어별 도구 > Swift / Apple” 섹션에서 설명됨)

이상으로 142개 ECC 스킬 전체를 안내해 드렸습니다.
궁금한 스킬이 있으시면 이름을 말씀해 주세요. 더 자세히 설명해 드리겠습니다.